University of Manouba
National School of Computer Science

ID:PCD/2425/50

Version:01

Report of the Design Project and Development

Topic: A Fine-Tuned Neural Network for Precise Brain Tumor Segmentation in Clinical MRI Analysis

Elaborated by :

Oumayma HAMMAMI Oussama CHAABANE Yasmine RIABI

Supervised by :

Dr. Marouene CHAIEB

Appreciations and Signature of the Supervisor

Appreciating my students' valuable contributions in this project. Wishing them continued success and good luck.

Dr. Marouene CHAIEB

Acknowledgments

We extend our sincere thanks and appreciation to Dr. Marouene CHAIEB, our supervisor, for his good directives, his availability and for his precious advice that he gave us in order to accomplish our project properly.

We extend our utmost respect and gratitude to the members of the jury for the honor they have done us in accepting to examine and evaluate this modest contribution, and to the administration and all the teaching staff of the ENSI for the training they have provided us during these two years.

We also express our gratitude to everyone who has contributed in one way or another to the good realization of the project.

Abstract

For our design and development project at ENSI, we aimed to explore the fields of AI and deep learning and contribute to healthcare, which often seems overlooked in technological advancements. Brain tumors, if not detected and treated early, can lead to severe neurological damage or even be fatal, making accurate and timely diagnosis a critical challenge in modern medicine. To address this gap, we developed a brain tumor segmentation model with the goal of achieving increased accuracy and reliability that can be effectively integrated into the healthcare sector.

Our first step involved selecting the most suitable model for our project. After careful consideration and experimentation, we adopted the nnU-Net framework, a self-configuring deep learning model known for its strong performance in biomedical image segmentation tasks. The training and evaluation of our model were conducted using the BraTS 2021 dataset, a widely recognized benchmark composed of multi-institutional pre-operative MRI scans, which is highly relevant for brain tumor segmentation studies.

The fine-tuning phase was particularly challenging as we sought to incorporate the strengths of existing models into nnU-Net, ultimately creating an efficient and reliable fine-tuned model for brain tumor segmentation. To ensure usability, we developed a simple and user-friendly interface targeted at healthcare professionals and other users. This interface accepts MRI inputs, analyzes the images, and determines whether a brain tumor is present. In the case of detection, the model focuses on analyzing the tumor for further insights.

In conclusion, our project adopts an innovative approach to brain tumor segmentation, aiming to advance this field, assist healthcare professionals, and contribute to early detection for timely treatment. The implementation code is publicly available at the following GitHub link:

https://github.com/oumaymahammami/mindscan.

Key Terms

Brain Tumor Segmentation, Medical Imaging, MRI Analysis, Healthcare AI Applications, Deep Learning, Convolutional Neural Networks, Transfer Learning, Fine Tuning.

List of Acronyms

MSA	Multi-Head Self-Attention
FNN	Feed-Forward Neural Network
LGG	Low-Grade Gliomas
MRI	Magnetic Resonance Imaging
HGG	High-Grade Gliomas
CNS	Central Nervous System
CNN	Convolutional Neural Network
AI	Artificial Intelligence
WT	Whole Tumor
TC	Tumor Core
ET	Enhancing Tumor

Contents

List of Figures

List of Tables

Chapter 1

Introduction

The brain is one of the most important organs in the human body and, along with other tumors of the CNS tumors, ranks as the fifth most common form of cancer, with more than 300,000 cases reported annually worldwide [4]. Gliomas are the most common primary brain tumors of the CNS and are categorized into two types:HGG and LGG.

MRI is a diagnostic technique that uses a large magnet, radio waves, and a computer to produce detailed images of structures within the human body. These images are typically black and white [5], making it hard for untrained eyes to distinguish between normal structures and abnormalities.

Our project aims to enhance an existing AI model to assist in the early diagnosis of brain tumors using MRI images.

1.1 Context of the Study

In recent years, deep learning and machine learning have revolutionized various fields, particularly computer vision and image analysis. Their integration into medical imaging has opened new possibilities for early and precise diagnosis [6]. However, despite this progress, there remains a noticeable gap in the accessibility and clinical application of these technologies for critical tasks

such as brain tumor segmentation. Many healthcare settings still lack robust and user-friendly tools that offer high accuracy in detecting and delineating tumors from MRI scans [7].

Artificial intelligence has the potential to significantly enhance diagnostic workflows by enabling automated and reliable pre-diagnosis of brain tumors. For instance, a study published in *Nature Medicine* demonstrated that a deep learning model, when combined with stimulated Raman histology, achieved a diagnostic accuracy of 94.6% in under three minutes, comparable to traditional methods that typically require 20–30 minutes [8]. Similarly, Yogananda et al. developed a fully automated deep learning network for brain tumor segmentation, achieving Dice scores of 0.90 for whole tumor, 0.84 for tumor core, and 0.80 for enhancing tumor on the BraTS dataset [9].

Despite these advancements, the deployment of such models in routine clinical practice remains limited. Challenges include the need for models that are not only accurate but also practical for real-world clinical use. Our project aims to address this gap by developing a deep learning-based solution tailored to clinical needs, combining precision with usability to support timely and effective medical decision-making.

1.2 Problem Statement and Motivation

Brain tumors present significant medical and economic challenges due to their high fatality rates and the complexities involved in their diagnosis and treatment. Traditionally, radiologists manually segment brain tumors, a process that is time-consuming, prone to human error, and costly. This manual segmentation becomes increasingly inefficient in light of advancements in medical imaging technologies, making it critical to explore automated techniques..

Furthermore, brain tumors present unique challenges due to their diverse

types, locations, and stages of development, requiring advanced approaches for accurate detection and management. The growing prevalence of brain tumors, along with advances in medical technology, intensifies the need for more effective diagnostic and therapeutic strategies. Research in automated brain tumor segmentation is vital to improving patient outcomes, reducing healthcare costs, and enhancing the quality of life for those affected by these life-threatening conditions. By addressing these challenges, deep learning-based segmentation methods could revolutionize the way brain tumors are diagnosed and treated, ultimately leading to better healthcare outcomes [10].

1.3 Aims and Objectives

Deep learning has shown promising results in image classification and segmentation, offering the potential to greatly improve the speed and precision of brain tumor detection, which is crucial for early diagnosis and effective treatment. Our project aims to leverage technological advancements in AI and deep learning to benefit the healthcare sector. We sought to address the challenges of brain tumor detection and segmentation by focusing our research efforts on fine-tuning an existing model for this purpose. To ensure the efficacy of our study, we defined the following objectives:

- Fine-tune a model with efficient and reliable accuracy.
- Provide a precise interpretation of the tumor surface.
- Develop an accessible interface for healthcare professionals to facilitate the detection of brain tumors.

1.4 Proposed Solution and Results

Brain tumor segmentation is a key challenge in medical imaging research. Recent advancements in deep learning have introduced powerful models capable of achieving high accuracy by leveraging multi-modal MRI datasets, such as BRATS. These approaches have significantly improved tumor detection and segmentation, aiding in diagnosis and treatment planning.

Our solution utilizes the **nnU-Net** framework, a state-of-the-art deep learning model designed for automated medical image segmentation [11]. nnU-Net was chosen for its flexibility, as it can adapt to various datasets without extensive manual configuration. It has been widely adopted in numerous medical imaging challenges due to its robustness and generalizability [12]. The framework efficiently handles the data preparation, preprocessing, model training, and evaluation processes, ensuring a streamlined workflow for accurate tumor segmentation. By using nnU-Net, we benefit from a highly modular and reproducible pipeline, which simplifies experimentation and boosts the reliability of the segmentation results in clinical settings.

In the evaluation phase, the performance of the model will be assessed using the Dice coefficient, a widely recognized metric for measuring the overlap between predicted segmentations and the ground truth. This metric will help ensure that the model delivers high-quality and reliable segmentation results.

By leveraging nnU-Net's automated capabilities and the multi-modal MRI data, our solution aims to provide reliable and precise tumor segmentation results. This approach minimizes the need for manual intervention, reduces errors, and enhances the efficiency of clinical workflows.

The evaluation of our solution demonstrated its ability to deliver high-quality segmentation performance. The model's automation, adaptability, and accuracy make it a valuable tool for medical imaging applications, con-

tributing to improved outcomes in brain tumor diagnosis and treatment.

1.5 Structure of the Report

This report delves into various aspects of our findings and contributions. For easier and proper comprehension of our study, the report is divided into sections, each addressing a different aspect of our project. The next section discusses the technical background necessary for our study, reviews the main related works in the literature and highlights the main findings of other studies. Section 3 will focus on the analysis and specification of requirements. Section 4 will present the conception and architecture of our project. Section 5 will outline the implementation workflow and present the final results. The report will conclude in Section 6, where we will discuss the possibilities for further improvement and development of the project.

Chapter 2

Literature Review

In this section, we begin by outlining the technical background necessary to understand the context of our study. Next, we review published research in the field of brain tumor segmentation, focusing on key methodologies and findings. Finally, we discuss the insights and limitations identified in the literature and highlight the contributions of our research in addressing these gaps.

2.1 Technical Background

For semantic segmentation tasks in medical imaging, Convolutional Neural Networks (CNNs) have traditionally been effective, especially when the dataset contains local features and has fewer learnable parameters. However, as the complexity of medical images increases, more advanced architectures are required to capture both local and global features effectively. In this regard, nnU-Net, a deep learning-based architecture, has become a popular choice for medical image segmentation tasks due to its flexibility, scalability, and outstanding performance across multiple types of image data.

The nnU-Net supports both 2D and 3D image segmentation, which is crucial for accurately analyzing brain tumor data that could be in either

format. The model consists of two main architectures: 2D U-Net and 3D U-Net. Both models are based on the U-Net structure, with the 3D U-Net being particularly suited for volumetric data. The flexibility of nnU-Net allows it to adapt to the problem at hand by adjusting its architecture, data preprocessing, and hyperparameters based on the specific dataset.

The nnU-Net's architecture consists of an encoder-decoder structure with skip connections. The encoder captures context and reduces the spatial resolution of the input image, while the decoder reconstructs the image back to its original size. This architecture allows for the effective extraction of both local and global features, which is essential for the accurate segmentation of complex structures like brain tumors. The network uses a combination of convolutions, normalization layers, and activation functions to ensure smooth learning and better generalization across different datasets.

In addition, nnU-Net's self-adaptive nature allows for automated configuration of hyperparameters, making it suitable for diverse datasets without requiring significant manual intervention. This feature has been particularly useful in our work, enabling us to focus on fine-tuning the model for brain tumor segmentation without having to worry about optimizing the architecture from scratch.

The following figure 2.1 comprises the structure of 3D U-Net.

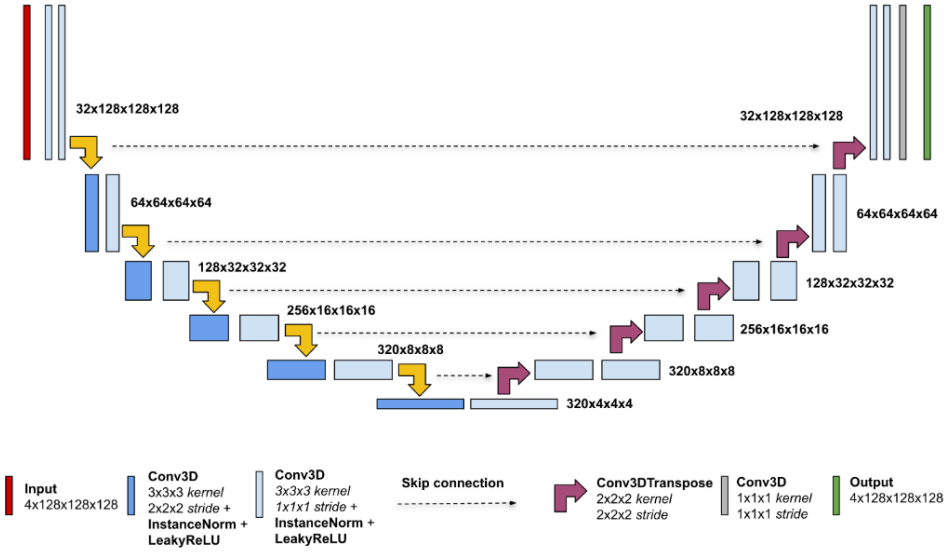


Figure 2.1: The 3D U-Net architecture [1]

The structure of nnU-Net comprises the following essential elements:

- **Image Patching and Embedding:** which is the first step in Vision Transformers consists of:
  - **Patch Splitting:** Dividing input images into fixed-size patches.
  - **Patch Flattening:** Flattening each patch into a 1D vector.
  - **Patch Embedding:** Using a learnable linear projection to project each flattened patch into an embedding dimension D. The linear transformation is designed to learn richer feature representations for each patch.
- **Positional Encoding:** are introduced to retain the spatial structure of the original image. This step involves adding encoded information about the location of each patch to its embedding. These encodings can either be learned during model training or predefined.

- **Transformer Encoder Layers:** are composed of two primary components: Multi-Head Self-Attention (MSA) and a Feed-Forward Neural Network (FFN) .

  Multi-Head Self-Attention (MSA) The self-attention mechanism allows each patch to attend to every other patch, modeling long-range dependencies and relationships between all parts of the image. Each patch computes an attention score, reflecting the weighted values of all other patches. This score is calculated as follows:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

  Where Q (query), K (key), and V (value) are learned linear projections of the input patch embeddings.
  - The dot product between queries and keys determines the attention score, which is normalized using softmax.
  - The weighted sum of values determines the output.

  Multi-head attention computes the attention mechanism in parallel across multiple attention heads, enabling the model to focus on different parts of the image simultaneously.

  Feed-Forward Neural Network (FFN): consists of two layers connected with a non-linear activation function. Each Transformer encoder layer incorporates residual (skip) connections and layer normalization.

- **Classification Token (CLS Token)** in segmentation tasks such as brain tumor segmentation, the CLS token is usually replaced by patch-level outputs. These outputs collectively contribute to the final segmen-

tation map, ensuring that detailed spatial and structural information is retained.

- **MLP Head (Multi-Layer Perceptron Segmentation Head):** which consists of upsampling layers and convolutional operations designed to transform patch-level outputs into a dense segmentation map. For brain tumor segmentation, these maps predict voxel-wise labels for different tumor regions, such as Whole Tumor (WT), Tumor Core (TC), and Enhancing Tumor (ET). A softmax or sigmoid activation function is applied for the final pixel level.

  Figure 2.2 illustrates the segmentation of a brain tumor, highlighting the Whole Tumor (WT), Enhancing Tumor (ET), and Tumor Core (TC). This image is taken from the research paper named Multi-step-Cascaded Networks for Brain Tumor Segmentation, which provides a detailed analysis of tumor subregions in the BraTS dataset.

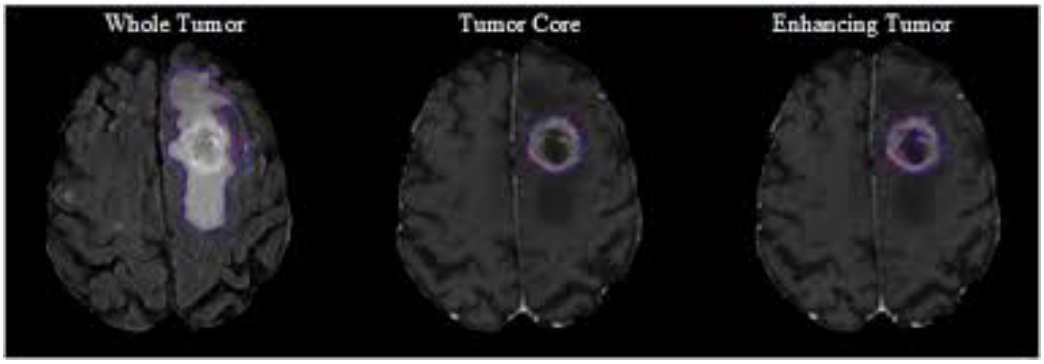


Figure 2.2: Segmentation-result of the whole tumor WT Tumor core TC and Enhancing tumor ET [2]

2.2 Related Works

In this section, we explore advances in detecting and segmenting brain tumors using magnetic resonance imaging, with a focus on machine learning and computer vision. In recent years, numerous studies have focused on fine-tuning existing Convolutional Neural Networks (CNNs) and Transformers to improve brain tumor detection. Most of these studies used the BraTS dataset from various years, which contains various 3D images. The dataset highlights different regions of brain tumors, including Whole Tumor (WT), Tumor Core (TC), and Enhancing Tumor (ET).

For brain tumor segmentation, researchers have primarily explored two approaches which are fine-tuning Transformers or combining CNNs and Transformers to mitigate the weaknesses of each individual model. In the following, we examine the methodologies and findings of each approach in detail.

2.2.1 Fine-Tuning Models

For example, in 2024, Zakariah et al. [13] introduced the Dual Vision Transformer-DSUNET model to improve brain tumor segmentation. Using the BRATS 2020 multi-modal MRI dataset, which includes T1-weighted, T2-weighted, T1Gd and FLAIR MRI modalities, the model achieved high performance: a Dice coefficient of 91.29% for all tumor classes, 99.93% training accuracy, and 99.93% segmentation accuracy. The model demonstrated exceptional precision and sensitivity, with values of 98.12% and 97.45% respectively, emphasizing its effectiveness in brain tumor diagnosis and treatment.

In 2024, Alzahrani and Qahtani [14] introduced a knowledge distillation framework using a tripartite attention mechanism (TriA) within transformer models for brain tumor detection from MRI scans, utilizing datasets with 15, 17, and 44 classes from three MRI modalities which are T1, enhanced con-

trast T1, and T2. The experimental results show that the TriA transformer outperforms other models, including Deep CNN, SA, NeiA, and Perceiver, in terms of accuracy and AUC, especially when using the AugPip augmentation technique. The student models, particularly those trained with the BTDS-15C dataset, demonstrate strong Top-1 and Top-5 accuracies, though they exhibit slightly higher loss compared to the teacher models.

In 2024, Yuan [15] conducted a study, presenting an enhanced approach to MRI brain tumor segmentation using an improved Mask RCNN model. The enhancements include a ResNet50 backbone combined with attention mechanisms (SENet and CANet) and a Feature Pyramid Network (FPN) to capture multi-scale tumor features effectively. RoIAlign ensures precise feature extraction by mapping irregular regions of interest to fixed dimensions, while data augmentation techniques address dataset limitations and improve model generalization. This model achieved good performance, with a Precision of 90.72% and Recall of 91.68%.

A study, realized by Li el al. [16] employs a transformer-based multi-task model using a Swin Transformer, designed for simultaneous glioma-infiltrated brain area classification and tumor segmentation from T1c and T2-FLAIR images. The Swin Transformer, a hierarchical vision transformer, captures both global context and local details, making it highly effective for complex medical imaging tasks. Its hierarchical architecture processes images at multiple resolutions, while window-based self-attention reduces computational cost by applying attention locally within image patches. Cross-patch information sharing ensures global dependencies are captured, crucial for understanding the tumor-brain relationship. Additionally, the model blends CNNs' local feature extraction strengths with transformers' global modeling capabilities. This approach achieved high performance, with a Dice Similarity Coefficient of 87.60%.

A research, realized by Ma el al. [17] introduces DTASUnet, a U-shaped

encoder-decoder model designed for 3D brain tumor segmentation using multimodal MRI inputs (T1, T2, T1ce, and T2-FLAIR). The model generates segmentation maps for three glioma sub-regions which are ET, WT, and TC. DTASUnet employs dual local and global transformers within its encoder, with local transformers learning short-range dependencies and global transformers capturing long-range context, enhanced by a pyramidal hierarchy for rich feature extraction. A 3D Channel and Spatial Attention Supervision module refines features by prioritizing critical channels and spatial locations, improving segmentation precision. While DTASUnet achieves significant performance gains, such as a 3.4% to 1.6% Dice score improvement on the BTCV dataset, it incurs higher computational costs, increasing validation time by 0.221 to 1.383 seconds.

2.2.2 Hybrid Models

In 2023, Lin et al. [18] proposed CKD-TransBTS, a hybrid model that combines a Swin Transformer and CNN (with MBConv layers). By leveraging the strengths of both architectures, the model addresses their individual limitations. Using the BraTS 2021 dataset, CKD-TransBTS achieved an average Dice score of 88.7% and an average accuracy of 90.4% for all tumor classes, outperforming many existing CNN and Transformer-based models.

Similarly, ZongRen et al. [19] proposed DenseTrans, which integrates Swin Transformer and CNN (UNet++) into a hybrid model. In DenseTrans, 3D CNNs are utilized for local feature extraction, while Swin Transformers handle long-distance dependency modeling and global context capture. This combination effectively enhances segmentation performance on the BraTS 2021 and BraTS 2020 datasets. DenseTrans achieved an average Dice score of 89.2% and an average accuracy of 91.1% on the BraTS 2021 dataset. On the BraTS 2020 dataset, it achieved an average Dice score of 88.7% for all tumor classes.

Zhang et al. [20] introduced AugTransU-Net, a model that combines a hierarchical CNN with enhanced Transformer modules. It incorporates Augmented Shortcuts and Paired Attention modules to capture long-range spatial and channel feature dependencies across network layers, achieving competitive performance on the BraTS 2019–2021 datasets. On the BraTS 2019-2021 datasets, AugTransU-Net reported an average Dice score of 83.4% and an average accuracy of 85.6% for all tumor classes.

In 2024, Fox et al. [21] introduced a novel Residual Vision Transformer (ResViT)-based self-supervised learning model for brain tumor classification, addressing the challenge of limited MRI datasets. The model achieves impressive accuracy, with 90.56% on the BraTs dataset, 98.53% on the Figshare dataset, and 98.47% on the Kaggle brain tumor dataset. By combining CNN and ViT in a hybrid architecture, the model outperforms traditional deep learning approaches, demonstrating a robust and efficient solution for MRI analysis and tumor classification.

The following table summarizes the results of fine-tuned transformers cited above, the table contains multiple metrics such as Dice Score (DS), 95% Hausdorff Distance (HD95), Sensitivity (Sn), Accuracy (Acc), Specificity (Sp), Precision (Pr), and F1-score (F1)

Table 2.1: Comparative Analysis of Deep Learning Models for Brain Tumor Segmentation on BraTS and BTDS Datasets

Research	**Model**	**Dataset**	**DS%**			**HD95 (mm)**			**Performances (%)**			
			ET	**TC**	**WT**	**ET**	**TC**	**WT**	**Sn**	**Acc**	**Sp**	**F1**
Zakariah et al.	DSUNET	BraTS 2020	**91.29**	**97.45**	–	–	–	–	**97.45**	**99.93**	**99.95**	**98.12**
Alzahrani and Qahtani	Tripartite Attention	BTDS-15C	–	–	–	–	–	–	–	94.07	–	–
Yuan et al.	Mask RCNN	–	–	–	–	–	–	–	90.73	–	89.96	91.24
	Mask RCNN+SE	–	–	–	–	–	–	–	91.62	–	90.31	91.69
	Mask RCNN+CA	–	–	–	–	–	–	–	91.68	–	90.72	91.85
Li et al.	Transformer-based multi-task	–	–	–	–	–	–	–	83.32	82.07	81.55	–
Ma et al.	DTASUnet + fix local transformer	–	85.5	86.0	–	–	–	–	–	–	–	–
	DTASUnet + shift local transformer	–	86.9	86.8	–	–	–	–	–	–	–	–
	DTASUnet + global transformer	–	85.3	86.1	–	–	–	–	–	–	–	–
	DTASUnet + dual local transformer	–	88.2	88.0	–	–	–	–	–	–	–	–
Lin et al.	CKD-TransBTS	BraTS 2021	88.5	91.03	92.46	5.93	9.36	**7.38**	89.97	–	–	94.74
ZongRen et al.	DenseTrans	BraTS 2020	82.3	85.3	91.4	15.2	**16.9**	6.32	–	–	–	–
		BraTS 2021	88.3	86.2	93.2	12.2	14.8	4.58	–	–	–	–
Zhang et al.	AugTransU-Net	BraTS 2019	78.2	80.4	89.7	4.11	9.56	6.65	–	–	–	–
		BraTS 2020	78.6	81.9	89.8	**24.31**	9.56	5.56	–	–	–	–
		BraTS 2021	87.8	91.8	**93.6**	8.01	6.16	5.88	–	–	–	–

2.3 Findings of the Literature and Contributions of the Study

This study sheds light on the current advancements in brain tumor segmentation by examining and synthesizing a wide range of recent literature. Our review of multiple deep learning models, including DSUNET, Transformer-based architectures, and variants like DTASUnet and AugTransU-Net, reveals significant progress in segmentation performance. However, several limitations persist. Many models are optimized for specific datasets such as BraTS or BTDS, which may restrict their generalizability to other clinical contexts. Some approaches prioritize segmentation accuracy but fall short in terms of computational efficiency or robustness across different tumor subregions (ET, TC, WT). Furthermore, few studies place enough emphasis on clinical applicability or user-friendliness for non-technical medical staff. These gaps indicate the need for models that are not only accurate but also adaptable, lightweight, and intuitive for use in real-world medical environments.

In response to these limitations, our study proposes a refined approach aimed at improving both the technical and practical aspects of brain tumor segmentation. Building on the insights gained from prior research, we have fine-tuned a model to enhance segmentation accuracy across all tumor components while maintaining computational efficiency. Unlike many existing models, our solution is designed with end-users in mind—particularly clinicians and healthcare professionals—emphasizing simplicity in usage and deployment. Our work also contributes by offering a more robust model capable of generalizing beyond a single dataset, thereby enhancing its utility in diverse clinical scenarios. By addressing the limitations identified in the literature, our study not only advances technical performance but also bridges

the gap between research innovation and practical application in the medical field.

Chapter 3

Analysis and Specification of Requirements

3.1 Requirement Analysis

The requirement analysis defines the actors, functional, and non-functional requirements to ensure the system meets user needs and operational goals. By identifying actors and their roles, we tailor the system to support effective brain tumor detection in MRI scans. The system will deliver necessary functionality, meet performance metrics like usability, availability, and data privacy, and provide an intuitive user experience for accurate and timely diagnoses.

3.1.1 Actors Identification

In the context of this system, the actors are individuals or entities that interact with the system to achieve certain tasks. The key actors identified in the system are:

- **Doctor**: The doctor uses the system to review MRI scans, analyze segmented tumor areas, adjust segmentation if necessary, and provide

a diagnosis. They are responsible for generating reports based on MRI results to aid in treatment decisions.

- **Scanner Agent** : The scanner agent is responsible for capturing MRI images of patients and uploading them into the system for further processing and analysis by the AI model. They send the images to the doctor once processed.

- **Admin**: The admin manages and maintains the equipment and systems used in the diagnostic process. They oversee user accounts, ensure smooth system operation, and monitor the performance of the AI model.

3.1.2 Functional Requirements

The functional requirements describe the specific tasks and actions each actor should be able to perform within the system. These requirements are outlined as follows:

- **Doctor**:
  - **Authentication**: The doctor must securely log in to the system to access patient data.
  - **Upload MRI Images**: The doctor should be able to upload MRI images for analysis.
  - **View AI-generated Tumor Segmentation**: The doctor must be able to view results from the AI model that segment tumor areas in the MRI images.

- **Scanner**:
  - **Authentication**: The scanner must securely authenticate to access the system.

  - **Upload MRI Images**: The scanner is responsible for uploading captured MRI images to the system for analysis by the AI model.

- **Admin**:

  - **Authentication**: The admin must securely log into the system.
  - **Manage Users**: The admin must have the ability to manage the list of users (scanners and doctors), adding, editing, or removing users as necessary.
  - **System Maintenance**: The admin is responsible for ensuring the smooth operation of the system, including performing regular maintenance tasks.
  - **Monitor AI Model Performance**: The admin must oversee the performance of the AI model, ensuring it is accurate and functioning correctly.

3.1.3 Non-Functional Requirements

The non-functional requirements focus on the system's performance, usability, and compliance to ensure a high-quality user experience and secure handling of data. These requirements are as follows:

- **Usability**:

  - The system should be designed with an intuitive interface and simple login process, ensuring ease of use for all actors (doctors, scanners, and admins).

- **Availability**:

  - The system must be available 24/7, ensuring that doctors can access patient data and monitor risks at any time, providing constant availability.

- **Performance**:
  - The system must be able to handle a large number of concurrent users without lag, errors, or performance degradation. It should maintain high performance even under heavy load.
- **Conformity**:
  - The system must comply with relevant health and data privacy regulations, such as HIPAA (Health Insurance Portability and Accountability Act) or GDPR (General Data Protection Regulation), to ensure patient confidentiality and secure data handling.

3.2 Needs of the Domain

The domain of brain tumor detection in medical imaging involves the use of advanced technologies, particularly artificial intelligence (AI) models, to enhance the accuracy and efficiency of diagnosis. The key needs of the domain include:

- **Annotated Datasets**: High-quality annotated datasets are critical for training AI models to accurately detect and segment brain tumors in MRI scans. The accuracy of AI models depends on the quality and variety of these datasets.
- **Data Privacy**: The system must follow stringent regulations regarding patient data to ensure confidentiality and security. This includes encrypting patient data and ensuring only authorized personnel can access sensitive information.
- **Real-Time Processing**: The system should be capable of processing MRI scans in real-time to support timely diagnosis. This is crucial for

ensuring that doctors can provide quick treatment options based on accurate results.

3.3 Requirements Specification

To define the functional requirements of our system, we utilize the modeling language to visually represent system interactions, making it easier to understand how different components work together.

3.3.1 Modeling Language

Our project adopts **UML** as the primary modeling language to describe system functionality and interactions. UML provides a structured approach to system design through various diagram types, ensuring clarity in communication between developers and stakeholders. It helps in capturing user requirements and defining how the system should behave.

3.3.2 Use Case Diagram

The **Use Case Diagram** represents the interactions between the system and its users. It highlights the main actors and their associated functionalities, providing an overview of the system's core operations. The diagram serves as a foundation for understanding user interactions and system workflows. **Figure 3.1** illustrates the use case diagram, offering a visual representation of these interactions.

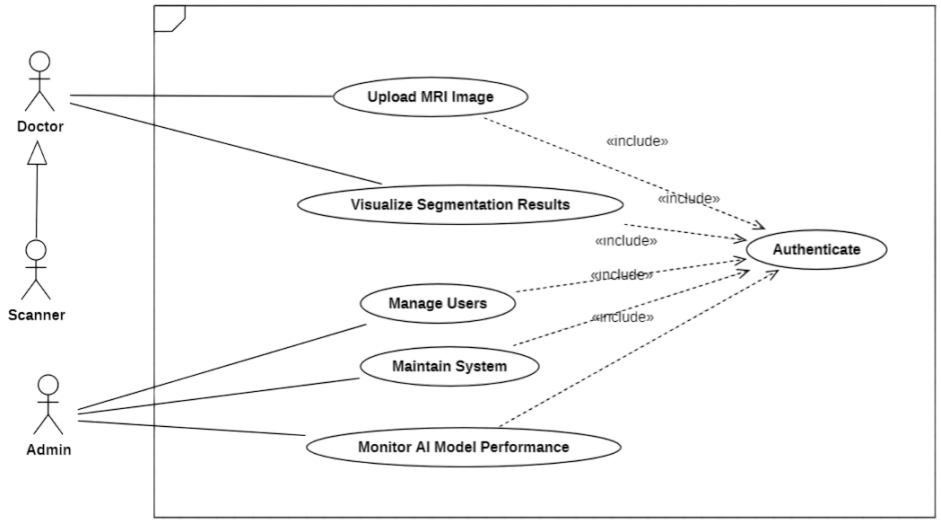


Figure 3.1: Use Case Diagram

3.3.3 Sequence Diagrams

A sequence diagram is a type of UML diagram that illustrates the flow of messages exchanged during interactions between objects. It presents a set of elements, represented by lifelines, along with the messages they exchange throughout their interaction. This particular sequence diagram represents a general execution scenario of the system.

The following diagram 3.2 is the sequence diagram for the authentication use case.

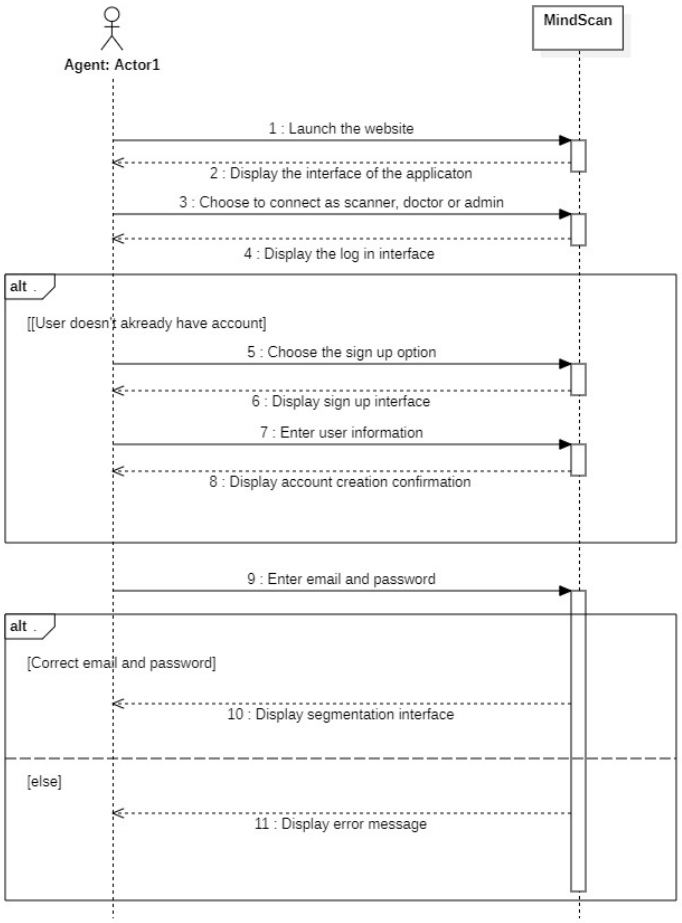


Figure 3.2: Sequence Diagram for the authentication use case

The following diagram 3.3 describe how the patient can view the segmentation results once he has uploaded the MRI Images

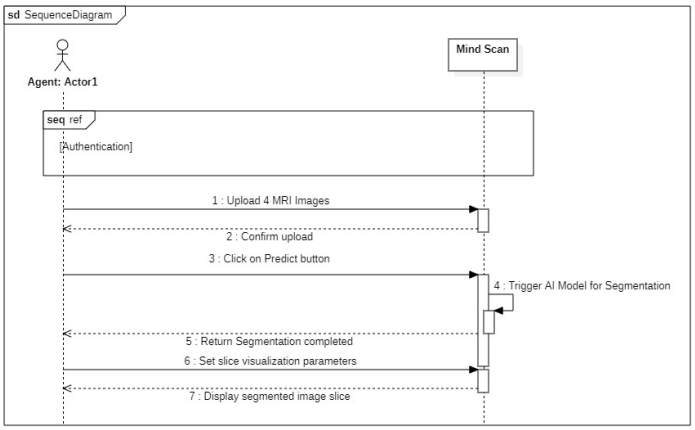


Figure 3.3: Sequence Diagram for view the results use case

Chapter 4

Design

To ensure a robust and scalable architecture, we have adopted the **Model-View-Controller (MVC) pattern** along with a **three-tier physical architecture**. This approach enhances modularity, maintainability, and scalability, making it easier to develop and manage the application.

4.1 MVC Model

The **Model-View-Controller (MVC)** pattern is a well-established architectural design that promotes the separation of application concerns. It divides the system into three distinct components:

- **Model:** Responsible for managing the application's data, business logic, and database interactions.
- **View:** Handles the presentation layer and defines how information is displayed to the user.
- **Controller:** Acts as an intermediary between the model and the view, processing user input and updating the model or view accordingly.

The decision to adopt the MVC pattern in our application stems from its numerous advantages:

- **Separation of concerns:** Improves code maintainability and scalability by isolating functionalities.
- **Reusability:** Enables independent reuse of components across the application.
- **Modularity:** Facilitates parallel development and easier integration of new features.
- **Testability:** Simplifies unit and integration testing through well-defined interfaces.

In the context of our brain tumor segmentation system, the MVC architecture is implemented as follows:

- **Model:**
  - Handles the processing of MRI images and the storage of segmentation results.
  - Interfaces with the **PostgreSQL** database to manage persistent data.
- **View:**
  - Developed using **Angular** to provide a responsive and user-friendly interface for visualizing results and interacting with the application.
- **Controller:**
  - Implemented with **Spring Boot**, it manages HTTP requests and orchestrates communication between the view and the model.

This structured approach, depicted in Figure 4.1, enhances the application's clarity, maintainability, and scalability by clearly delineating each layer's responsibility.

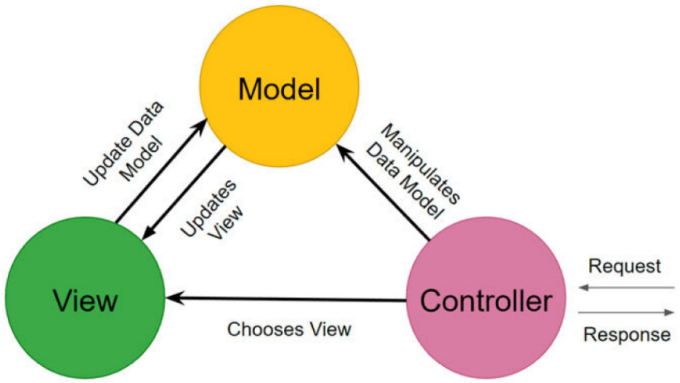


Figure 4.1: MVC Architecture

4.2 Physical Architecture

The system follows a **three-tier architecture**, a widely adopted design pattern for distributed applications. This architecture promotes separation of concerns by organizing the system into three distinct layers:

- **Presentation Tier:** The front-end interface through which users interact with the system, typically via a web browser.
- **Logic Tier:** The middle layer responsible for processing data, enforcing business rules, and coordinating application workflows.
- **Data Tier:** The back-end layer responsible for data persistence and retrieval.

In our brain tumor segmentation system, the three-tier architecture is realized as follows:

- **Presentation Tier:**
  - Built using **Angular** to deliver a responsive and user-friendly interface.
  - Communicates with the application backend through HTTP requests.
- **Logic Tier:**
  - Implemented using **Spring Boot**, exposing RESTful APIs to manage business logic, user actions, and image processing workflows.
- **Data Tier:**
  - Utilizes **PostgreSQL** as the relational database system for storing MRI analysis results, user credentials, and metadata.

This architectural design ensures a clear separation of responsibilities, enhances system scalability, facilitates independent development and deployment of each tier, and supports long-term maintainability. The overall structure is depicted in Figure 4.2.

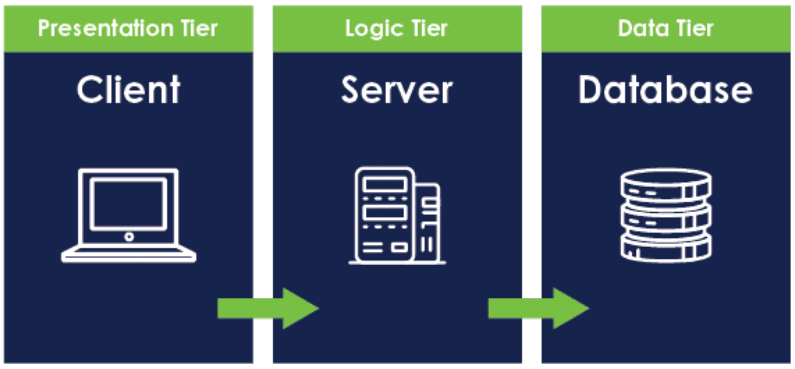


Figure 4.2: Three-Tier Architecture of the System

4.3 Detailed Conception

The detailed conception phase of system design focuses on providing a clear and structured overview of the system's architecture and components. It involves creating high-level models such as package and class diagrams to represent the various layers, classes, and their interactions.

4.3.1 Object Sequence Diagram

In this section we will focus on the most important sequence diagrams. The following Sequence Diagram 4.3 represents the sequence diagram for the "Upload MRI Images" use case.

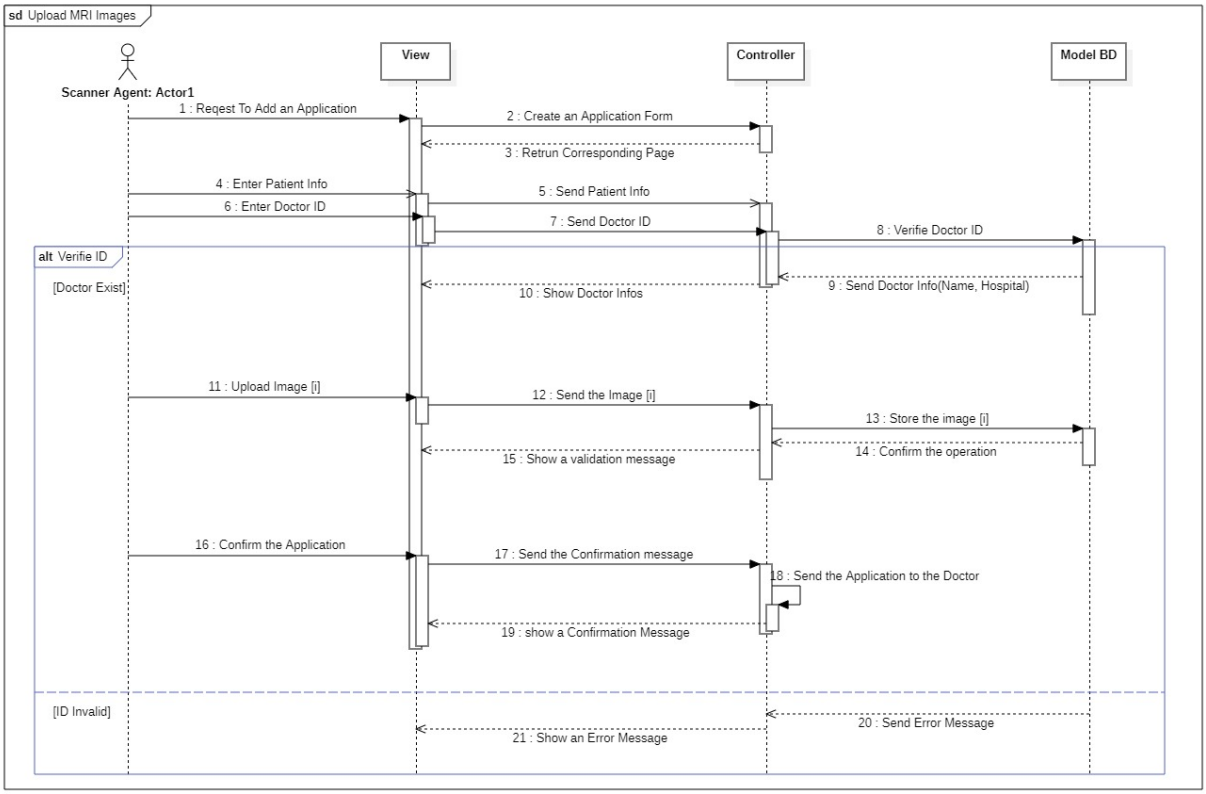


Figure 4.3: Sequence Diagram 1

This Sequence Diagram 4.4 demonstrates the detailed steps that the doctor needs to follow to use our model and segment brain tumor images.

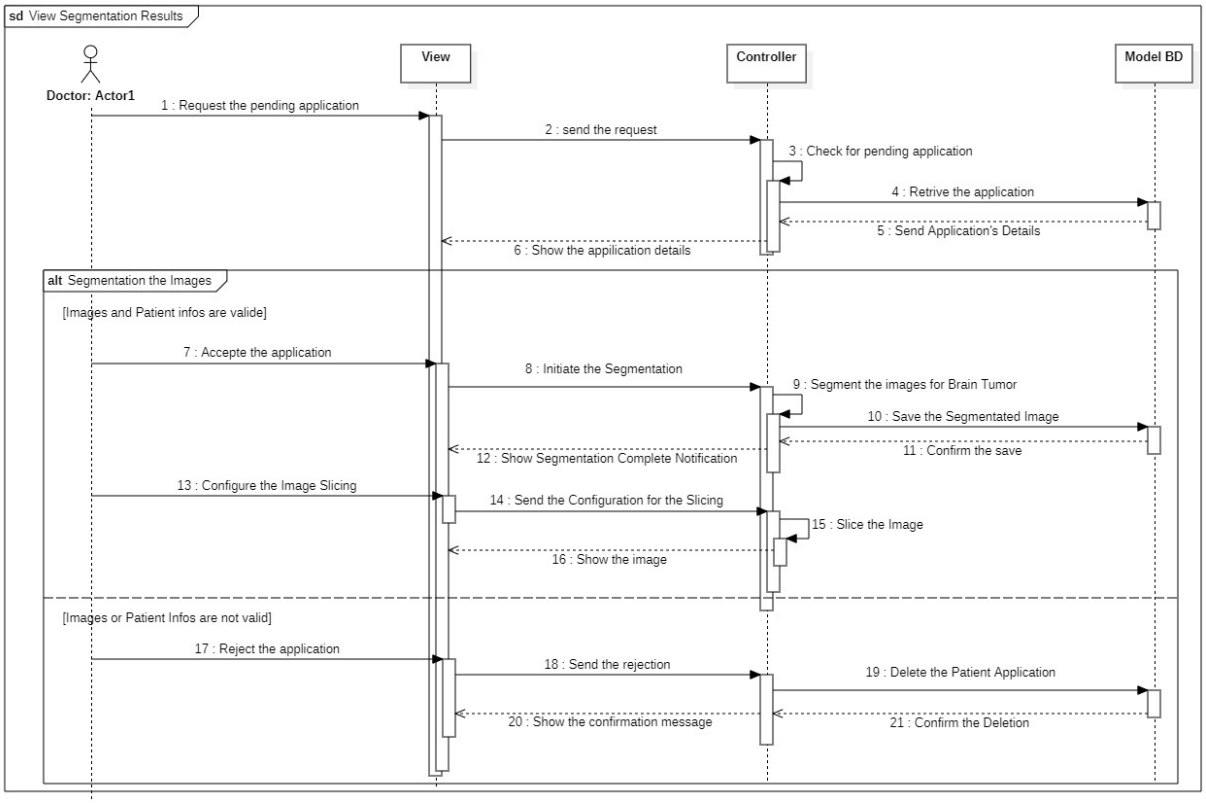


Figure 4.4: Sequence Diagram 2

4.3.2 Activity Diagram

An activity diagram visually maps out a process from its initiation to its completion. It details the sequence of actions and explicitly presents all potential decision points, illustrating the various paths the process can take as events unfold. Our activity diagram is shown in 4.5

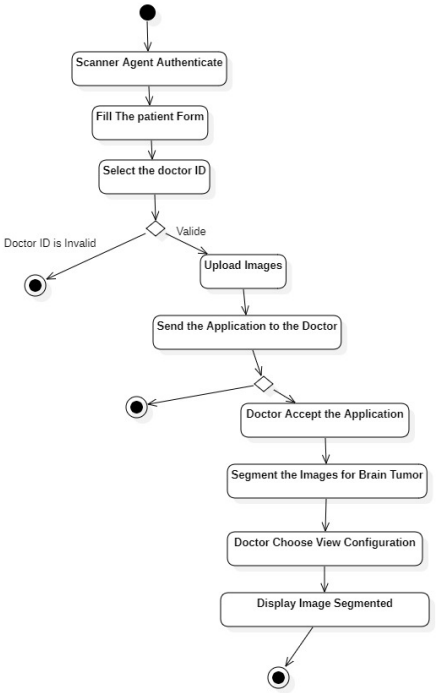


Figure 4.5: Activity Diagram

4.3.3 Package Diagram

This UML package diagram illustrates a structured and modular system architecture, divided into four key layers. The domain layer encompasses the Presentation Layer and User Interface, which manage user interactions and display information. The data layer includes Data Access and Service Agents, handling data operations and integrations with external services. The data sources layer represents the Database, serving as the primary repository for application data. Additionally, the cross-cutting layer provides shared functionalities such as Monitoring and Communication and messaging across all components. This layered UML design promotes separation of concerns, scalability, and maintainability, ensuring a robust and adaptable system. Figure 4.6 illustrates the UML package diagram and its architectural layers.

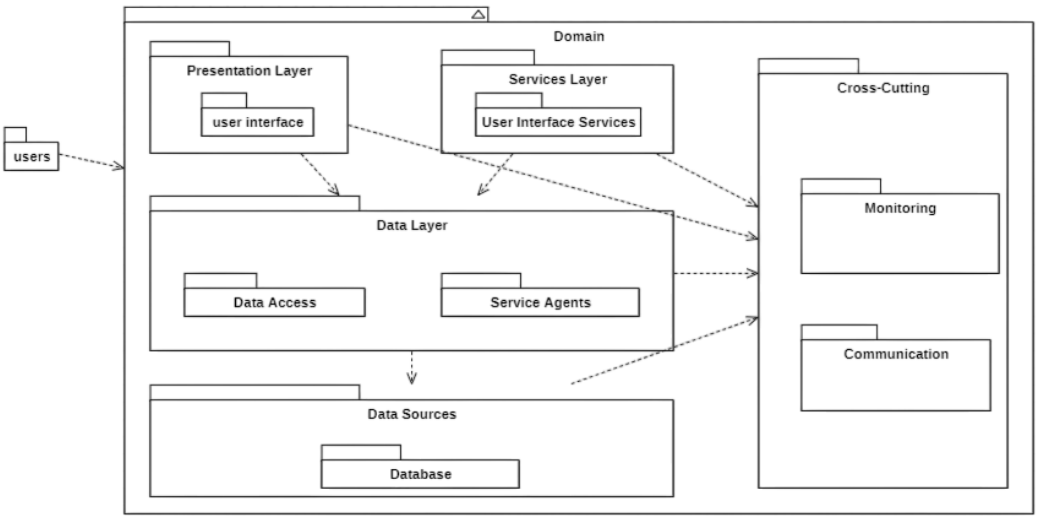


Figure 4.6: Package Diagram

4.3.4 Class Diagram

In our system design, we employed a UML class diagram to visualize the architecture, relationships, and functionalities of key components. The diagram comprises essential classes such as `Admin` and `User`, which handle authentication with attributes like `name`, `email`, and `password`. The `MRI Image` class stores image data, while the `Processing Unit` executes critical operations like image processing and result evaluation. Medical professionals interact through the `Doctor` class, enabling MRI image uploads and segmentation result reviews, while the `Scanner` class facilitates image capture and uploads. The diagram also specifies relationships, such as the one-to-many association between `Admin` and `User`, illustrating administrative oversight of multiple accounts. This structured representation ensures clarity in system design, facilitating efficient development and scalability. Figure 4.7 illustrates the

UML class diagram with its main components and associations.

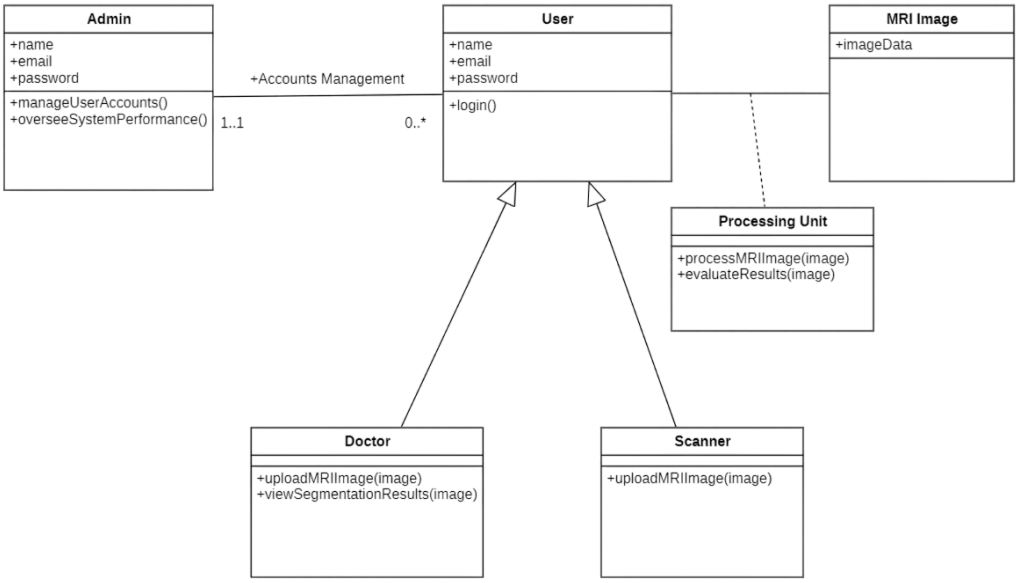


Figure 4.7: Class Diagram

Chapter 5

Methodology of the study

The methodology of the study chapter outlines the step-by-step process used to conduct the research. It begins by describing how the data was prepared, including preprocessing techniques like resampling and normalization. The chapter then explains the model architecture, focusing on the U-Net-based approach used for segmentation tasks. Key aspects such as data augmentation, hyperparameter tuning, and evaluation methods are also discussed. This chapter provides a comprehensive understanding of how the study was structured and the techniques employed to ensure reliable and accurate results in medical image segmentation.

5.1 Data Collection and Understanding

The following Figure 5.1 provides an overview of the complete pipeline used for brain tumor segmentation. The nnU-Net framework automatically adapts its architecture, preprocessing, and training strategy to the specific characteristics of the dataset, ensuring robust performance across various medical imaging tasks.

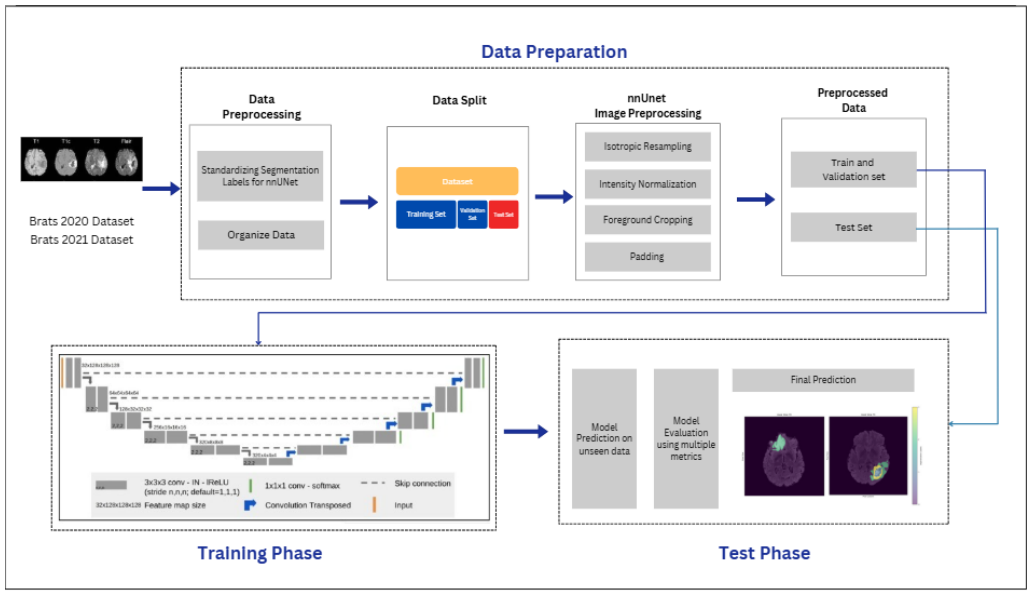


Figure 5.1: Overview of the methodology for brain tumor segmentation using nnU-Net.

The dataset utilized in this project is the Brain Tumor Segmentation (BraTS) 2021 dataset, a renowned benchmark dataset developed specifically for brain tumor segmentation challenges. BraTS 2021 is an integral part of ongoing efforts to advance medical image analysis using deep learning methods, with a particular focus on glioblastoma, a malignant and aggressive form of brain cancer. This dataset comprises a diverse and extensive collection of multi-parametric magnetic resonance imaging (mpMRI) scans, providing crucial insights into the anatomical structure of the brain and the subregions of brain tumors.

The BraTS 2021 dataset includes multiple types of MRI sequences: T1-weighted (T1), contrast-enhanced T1-weighted (T1ce), T2-weighted (T2), and fluid-attenuated inversion recovery (FLAIR) sequences. Each of these modalities captures distinct tissue contrasts, helping differentiate between healthy brain tissue and various tumor subregions. Specifically:

- **T1-weighted scans** provide structural details of the brain.
- **T1ce scans** highlight contrast-enhanced areas, indicating active tumors.
- **T2-weighted scans** reveal edema or swelling around the tumor.
- **FLAIR sequences** are sensitive to abnormal tissue, especially areas with edema or necrosis.

By combining these four modalities, the dataset enables precise segmentation of tumor regions.

The dataset contains 1350 cases (6750 individual mpMRI scans), a significant increase from BraTS 2020, which included only 660 cases (2,640 scans). This expansion enhances dataset diversity, improving the robustness of segmentation models trained on it. The increased dataset size allows for a more comprehensive analysis of glioblastoma, providing an accurate representation of tumor variations in shape, size, and behavior.

A key objective of the BraTS 2021 challenge is the segmentation of three critical tumor subregions:

- **Necrotic core:** The central tumor region with dead cells.
- **Peritumoral edema:** The swollen area around the tumor caused by inflammation.
- **Enhancing tumor:** The actively growing region that shows contrast enhancement.

The following image 5.2 shows the three tumor subregions: necrotic in blue, edema in green, and enhancing tumor in yellow:

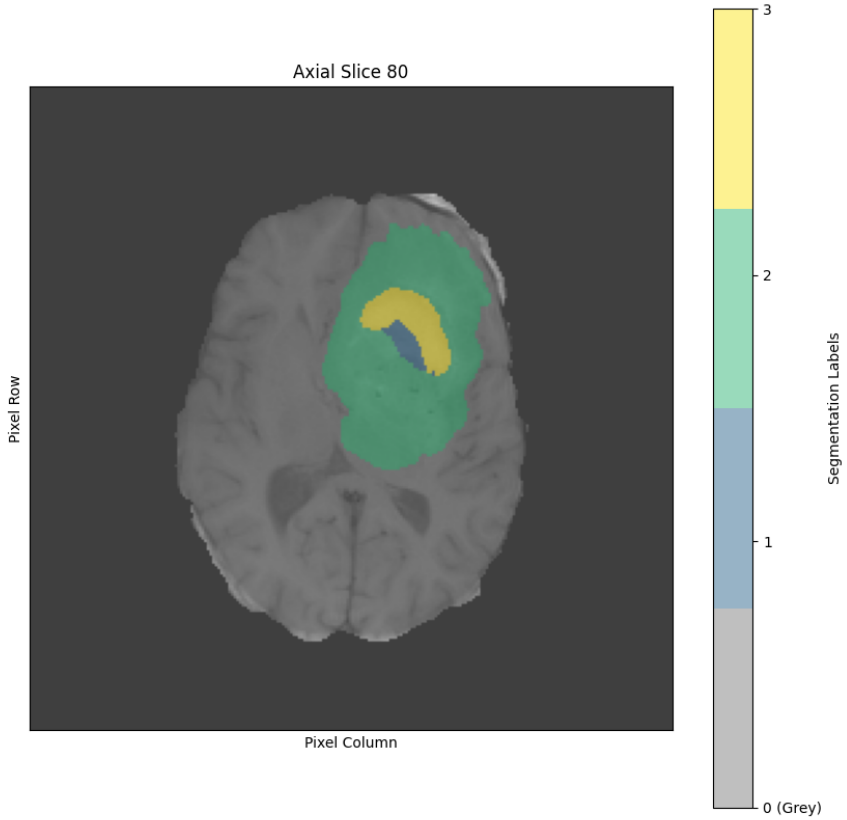


Figure 5.2: Three subregions of the tumor

These subregions are crucial for understanding tumor progression and its impact on surrounding brain tissue. Expert radiologists annotate the dataset, ensuring high-quality ground truth segmentations essential for training deep learning models. These expert annotations allow for precise model evaluation and clinical validation.

The dataset is collected from multiple institutions, ensuring diversity and

real-world applicability. Data sources include:

- **The Cancer Imaging Archive (TCIA)**
- **TCGA-GBM** (The Cancer Genome Atlas Glioblastoma)
- **TCGA-LGG** (Low-Grade Glioma dataset)
- **IvyGAP** (Ivy Glioblastoma Atlas Project)

These sources ensure a broad representation of glioblastoma cases with different genetic and phenotypic characteristics. Figure 5.3 shows examples of MRI scans from the BraTS 2021 dataset that highlight various tumor characteristics and their impact on brain structure.

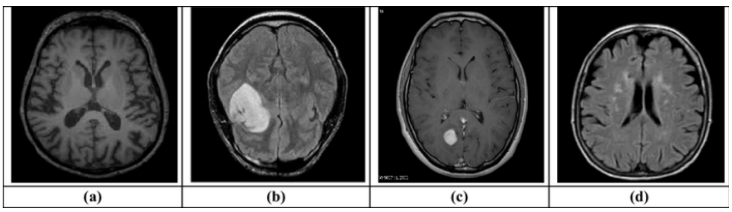


Figure 5.3: Sample MRI of brain tumor segmentation from the BraTS21 Challenge 2021 dataset [3]. (a) Normal, (b) Meningioma, (c) Pituitary, and (d) Glioma.

5.2 Dataset Organization

The dataset follows `nnU-Net` framework conventions, facilitating seamless integration into deep learning models. Key folders include:

- **"imagesTr"** – Training images.
- **"labelsTr"** – Ground truth segmentation labels.
- **"imagesTs"** – Test images for model evaluation.

- **"dataset.json"** – Metadata file containing dataset structure, imaging modalities, and label definitions.

5.3 Data Preprocessing

Data preprocessing is a crucial step in preparing a dataset for deep learning model training, particularly when working with medical imaging data like the BraTS 2021 dataset. It ensures that the data is standardized, consistent, and free from potential biases, all of which are essential for achieving accurate model performance.

The preprocessing process involves several essential steps, specifically:

- **Standardizing segmentation labels**: Segmentation labels are standardized to align with the nnU-Net framework, which ensures consistency across the dataset. This includes converting tumor subregion labels to match expected class indices (edema, enhancing tumor, and necrotic core).

- **Organizing the dataset**: The dataset is structured according to nnU-Net requirements, with training and testing images separated, files named according to modality (T1, T2, FLAIR), and segmentation masks stored in a dedicated folder.

- **Generating configuration files**: nnU-Net automatically generates configuration files containing metadata such as modality, number of classes, and label values.

These preprocessing steps are essential for preparing the dataset for use in deep learning models and are automated to enhance efficiency and reproducibility [22].

5.4 Model Building

Model building is a crucial phase in developing a deep learning pipeline, particularly for medical image segmentation tasks. In this study, we employ a U-Net-based architecture, specifically the nnU-Net framework, which is widely recognized for its adaptability and performance in medical imaging challenges such as brain tumor segmentation.

This section outlines the model building process in detail, focusing on data preparation, architectural components, and design considerations.

5.4.1 Data Preparation for Model Input

After preprocessing, the dataset is prepared to meet the requirements of the neural network. The following steps are applied to ensure consistency and compatibility:

- **Isotropic Resampling:** Ensures uniform voxel spacing across all input images.
- **Intensity Normalization:** Standardizes intensity values for consistency across scans.
- **Foreground Cropping:** Removes non-informative background regions to reduce input size.
- **Padding:** Ensures all inputs conform to the expected input shape of the network.

5.4.2 Dataset Splitting

To enable training, validation, and evaluation of the model, the dataset is divided as follows:

- **Training Set:** Used to learn model parameters.
- **Validation Set:** Assists in hyperparameter tuning and monitoring overfitting.
- **Test Set:** Evaluates the model's generalization performance.

5.4.3 Neural Network Architecture

The model architecture is based on a 3D U-Net-like structure, adapted through the nnU-Net framework to best suit the input data characteristics.

Key components of the architecture include:

- **Convolutional Layers:** 3×3×3 convolutions extract hierarchical spatial features from 3D MRI volumes.
- **Instance Normalization and ReLU:** Applied after each convolution to stabilize training and introduce non-linearity.
- **Encoder-Decoder Path:** The encoder extracts contextual information via downsampling, while the decoder recovers spatial resolution through upsampling.
- **Skip Connections:** Connect corresponding layers in the encoder and decoder paths to preserve spatial detail.
- **Output Layer:** A softmax activation provides probabilistic outputs for multi-class segmentation of tumor subregions.

This architecture enables the segmentation of complex tumor structures, such as the enhancing tumor, edema, and necrotic core, by leveraging both global context and fine-grained spatial information.

5.5 Training Strategy

The training strategy begins with setting up the environment and the preprocessing pipeline necessary to prepare the dataset for model training. Key steps in this pipeline include library installation, data download and extraction, dataset organization, and data visualization to ensure the data is ready for use in model training.

5.5.1 Installation of Required Libraries

The preprocessing pipeline starts by setting up the necessary libraries and tools that enable efficient handling of the dataset:

- **nnU-Netv2**: This framework automates essential preprocessing tasks, including image normalization, resampling, and augmentation, ensuring the dataset is consistently prepared with minimal manual intervention.
- **gdown**: A library used for downloading the dataset directly from Google Drive, ensuring seamless access to the raw data for preprocessing.
- **Supporting Libraries**:
  - **NumPy and Pandas**: Fundamental libraries for numerical operations and data manipulation, crucial for organizing and managing the dataset.
  - **NiBabel**: A library designed for reading and writing MRI data in NIfTI format, necessary for handling brain imaging data.
  - **OpenCV and scikit-image (skimage)**: These libraries provide powerful tools for image manipulation, such as resizing, rotation,

and other transformations needed for preparing MRI scans in various resolutions and orientations.

5.5.2 Dataset Preparation and Organization

The BraTS 2021 dataset is downloaded from a Google Drive link and extracted into a designated directory, ensuring availability for processing. Unnecessary files or folders are removed to maintain an organized workspace.

The dataset is structured into subdirectories for efficient training and testing:

- **imagesTr**: Contains training images in .nii.gz format.
- **imagesTs**: Contains test images.
- **labelsTr**: Stores corresponding ground truth label files for training images.

A `dataset.json` file is created to store important metadata, including:

- Dataset name, description, reference, and license information.
- Imaging modalities used (T1, T1ce, T2, FLAIR).
- Classes and labels (e.g., background, necrotic tissue, edema, and enhancing tumor regions).
- The number of training and test cases, with paths to the images and labels.

This process ensures proper organization and enables the model to effectively interpret the images and labels. Figure 5.4 illustrates an example MRI slice visualization, showing how the images are organized for model interpretation.

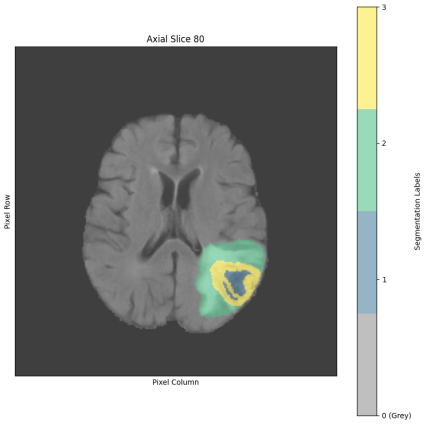


Figure 5.4: Example MRI slice visualization.

- **Class 0:** Background
- **Class 1:** Necrotic (NCR) & Non-Enhancing Tumor (NET)
- **Class 2:** Peritumoral Edema (ED)
- **Class 3:** Enhancing Tumor (ET)

5.5.3 Model Building

Model building is a crucial phase in developing a deep learning pipeline, especially for medical image segmentation tasks. In this study, we employ a U-Net-based architecture, specifically the nnU-Net framework, which is renowned for its adaptability and performance in medical imaging challenges, such as brain tumor segmentation.

This section outlines the model building process, including data preparation, neural network architecture, and the approach used for brain tumor segmentation.

5.5.3.1 Data Preparation for Model Input

After preprocessing, the dataset is prepared to meet the neural network's requirements. The following steps are applied to ensure consistency and compatibility:

- **Isotropic Resampling**: Ensures uniform voxel spacing across all input images.
- **Intensity Normalization**: Standardizes intensity values for consistency across scans.
- **Foreground Cropping**: Removes non-informative background regions to reduce input size.
- **Padding**: Ensures all inputs conform to the expected input shape of the network.

5.5.3.2 Dataset Splitting

The dataset is divided into three subsets to enable training, validation, and evaluation:

- **Training Set**: Used to learn model parameters.
- **Validation Set**: Assists in hyperparameter tuning and monitoring overfitting.
- **Test Set**: Evaluates the model's generalization performance.

5.5.3.3 Neural Network Architecture

The model is based on a 3D U-Net-like structure, adapted through the nnU-Net framework to best suit the input data characteristics. Key components of the architecture include:

- **Encoder Path**: Extracts hierarchical features.
- **Bottleneck**: Captures high-level abstract representations.
- **Decoder Path**: Reconstructs a full-resolution segmentation map.

The nnU-Net framework introduces automated adjustments and optimizations based on the dataset's characteristics, making it a self-configuring model tailored for medical image segmentation tasks.

5.5.3.4 Encoder Path – Feature Extraction

The encoder progressively extracts features from input MRI scans while reducing spatial resolution. This process helps the model learn different levels of abstraction, from low-level textures to high-level tumor structures.

Key Parameters Used in the Encoder Path

- **Input Image Size**: 3D full resolution – nnU-Net automatically resizes images to maintain the best trade-off between computational efficiency and detail preservation.
- **Input Channels**: 1 (grayscale MRI scans) – since our dataset consists of single-channel images.
- **Base Number of Filters**: Dynamically determined – nnU-Net adjusts this based on dataset characteristics to ensure optimal feature extraction.
- **Number of Encoding Levels (Depth)**: Adapted based on image resolution – more depth is added if higher resolution is detected.
- **Layer Types**: 3D Convolutions → Normalization → ReLU Activation
- **Kernel Size**: Varies per stage – nnU-Net tunes the kernel size at different depths for efficient feature capture.

- **Downsampling Strategy**: Pooling Operations – Reduces spatial dimensions to capture multi-scale contextual information.
- **Attention Gates**: Enabled – Focuses on important tumor regions while suppressing irrelevant background details.
- **Normalization**: Instance Normalization – Helps stabilize training and improve feature distribution.

5.5.3.5 Bottleneck – Compressed Feature Representation

The bottleneck serves as the bridge between the encoder and decoder. It compresses extracted features into a compact representation, filtering out irrelevant information while preserving crucial details.

Key Parameters Used in the Bottleneck

- **Number of Filters**: Optimized dynamically – Ensures that enough information is retained without unnecessary complexity.
- **Layer Types**: 3D Convolutions → Normalization → ReLU Activation
- **Activation Function**: ReLU – Introduces non-linearity to help capture complex tumor structures.
- **Loss Function**: Memory Efficient Soft Dice Loss – Prioritizes shape-based segmentation accuracy while reducing computational overhead.
- **Deep Supervision**: Enabled – Intermediate segmentation maps at multiple scales help stabilize training and improve accuracy.
- **Automatic Regularization**: nnU-Net applies adaptive regularization to prevent overfitting.

5.5.3.6 Decoder Path – Segmentation Reconstruction

The decoder reconstructs the full-resolution segmentation map by progressively upsampling the bottleneck representation while integrating details from earlier encoding stages.

Key Parameters Used in the Decoder Path

- **Upsampling Strategy**: Transposed Convolutions – Restores spatial resolution while preserving learned features.
- **Skip Connections**: Enabled – Connects corresponding encoder layers to the decoder to retain fine-grained spatial information.
- **Number of Decoding Levels**: Matches the encoder depth – Ensures symmetry in feature reconstruction.
- **Layer Types**: 3D Convolutions → Normalization → ReLU Activation
- **Post-Processing**: Largest Component Selection – Filters out false positives by keeping only the most significant tumor region.

5.6 Evaluation Metrics

To assess the performance of our brain tumor segmentation model, we compared the predicted segmentation masks with the ground truth labels using several quantitative evaluation metrics. These metrics provide a comprehensive understanding of the model's accuracy and reliability across different aspects of segmentation performance:

- **Dice Similarity Coefficient (DSC)**: Measures spatial overlap between the predicted segmentation (S) and the ground truth (G). It is defined as:

$$DSC = \frac{2|S \cap G|}{|S| + |G|}$$

A score of 1 indicates perfect segmentation, while a lower score reflects greater mismatch.

- **Intersection over Union (IoU)**: Evaluates the accuracy of the segmentation by calculating the ratio between the intersection and union of S and G:

$$IoU = \frac{|S \cap G|}{|S \cup G|}$$

- **Precision**: Represents the proportion of correctly predicted tumor voxels among all predicted positives:

$$Precision = \frac{TP}{TP + FP}$$

- **Recall**: Indicates the proportion of actual tumor voxels that were correctly identified:

$$Recall = \frac{TP}{TP + FN}$$

  Where:

  - **TP (True Positives)**: Correctly segmented tumor voxels.
  - **FP (False Positives)**: Non-tumor voxels incorrectly predicted as tumors.
  - **FN (False Negatives)**: Tumor voxels missed by the model.

- **Cross-Validation**: We applied 5-fold cross-validation to assess the model's generalization performance. The dataset was split into five parts; in each fold, the model was trained on four parts and tested on the remaining one. This approach improves robustness and reduces the risk of overfitting.

5.7 Experimenting with Different Configurations

nnU-Netv2 offers a range of configurable options, allowing us to fine-tune key parameters such as the number of epochs, architecture, and fold selection. To optimize our model, we conducted experiments using both the BraTS 2020 and BraTS 2021 datasets, testing different configurations:

- **Number of epochs:** 100 and 250
- **Cross-validation:** 5-fold
- **Architecture:** Standard nnU-Net and ResEnc nnU-Net

To further enhance our results, we explored a combination of outputs from both architectures. Each architecture generates an .ngz file containing a probability map for each class. Our approach involved two methods:

- **Max Probability:** For each voxel in the image, we selected the class with the highest probability from the two maps using the simple formula:

$$C_{\text{final}}(v) = \arg \max_{c \in \{0,1,2,3\}} (P_{\text{standard}}(v, c), P_{\text{ResEnc}}(v, c)) \tag{5.1}$$

  where:

  - $C_{\text{final}}(v)$ represents the final assigned class for voxel v.
  - $P_{\text{standard}}(v, c)$ and $P_{\text{ResEnc}}(v, c)$ denote the probability of voxel v belonging to class c, as predicted by the standard nnU-Net and ResEnc models, respectively.
  - $\arg\max$ selects the class c that has the highest probability across both models.

- **Probability Averaging:** We averaged the probabilities from both output maps before applying the softmax function to determine the final classification for each voxel, using this formula :

$$C_{\text{final}}(v) = \arg \max_{c \in \{0,1,2,3\}} \left(\frac{P_{\text{standard}}(v, c) + P_{\text{ResEnc}}(v, c)}{2} \right)$$

These strategies were designed to improve the segmentation accuracy, enhance metric results, and produce smoother and more reliable segmented images.

Chapter 6

Implementation and Results

This chapter presents the practical implementation of the proposed brain tumor segmentation model. It outlines the technical setup, training procedure, and evaluation process used throughout our experiments. We detail the configuration of the environment, the preprocessing steps, and the strategies applied to train and fine-tune the model. Furthermore, the chapter highlights the results obtained from various experiments, emphasizing the model's performance based on key evaluation metrics. Finally, we discuss challenges encountered during the implementation phase and how they were addressed.

6.1 Environment and Working Tools

To ensure the smooth execution and optimal performance of the project, a well-structured environment integrating both hardware and software components was established. This section presents an overview of the technological setup, including the specifications of the hardware used and the software tools that supported development and execution.

6.1.1 Hardware Environment

The hardware environment provides the necessary computational resources to support tasks such as AI model training, data processing, and application deployment. In the following, we present the main hardware configurations used in this study:

- **High-Performance Server (provided by CCK)**:
  - Machine Protocol: RDP
  - vCPU: 64 virtual CPUs
  - RAM: 256 GB
  - Disk Storage: 1000 GB
  - vGPU: A licensed vGPU is activated to enhance graphical processing capabilities, especially for deep learning tasks.
- **Work Units:** These units were used to distribute workloads and assist in various development and testing tasks. The specifications are as follows:

Work Unit	Processor	RAM	Graphics Card
1	13th Gen Intel® Core™ i5-13450HX (2.40 GHz)	16 GB	NVIDIA RTX 3050
2	11th Gen Intel® Core™ i5-11320H (3.19 GHz)	16 GB	NVIDIA GTX
3	11th Gen Intel® Core™ i5-11320H (2.60 GHz)	16 GB	NVIDIA GTX

Table 6.1: Hardware Specifications per Work Unit

6.1.2 Software Environment

To complement the hardware infrastructure, a variety of software tools and technologies have been selected for development and execution. This section outlines the software stack used in the project, including programming languages, frameworks, databases, and development tools that support the system's functionality.

6.1.2.1 Programming Languages

The choice of programming languages is crucial in determining the performance, maintainability, and scalability of the project. Each language has been selected to handle specific tasks, ensuring optimal performance across both the frontend and backend.

- **Python**: Used for machine learning and AI model development. Its simplicity and flexibility make it ideal for developing complex algorithms.
- **Java**: Used for the backend, Java offers a robust, scalable solution that is widely trusted in enterprise applications. It's a solid choice for creating high-performance, reliable server-side components.
- **TypeScript**: A superset of JavaScript, TypeScript is used alongside Angular to provide strong typing, making the code more reliable and easier to maintain. It's essential for developing dynamic and well-structured web applications.

6.1.2.2 Frameworks and Libraries

Frameworks and libraries are key to speeding up development by providing pre-built solutions for common tasks. We've integrated a mix of powerful

tools to streamline the entire development process, allowing us to focus on building innovative features.

- **Angular**: A TypeScript-based frontend framework used for building dynamic and responsive web applications. It provides a component-based architecture, making it easier to develop and maintain complex user interfaces.

- **Spring Boot**: A Java framework that simplifies backend development by offering pre-configured setups for building and deploying RESTful APIs. It handles user authentication and authorization, providing a secure and scalable solution for login, registration, and session management.

- **matplotlib**: A library for creating visualizations, used for plotting data and images, especially useful for displaying results during model evaluation.

- **PyTorch**: Provides powerful tools for building neural networks, performing tensor computations, and accelerating model training using GPUs.

- **SimpleITK**: A library specifically designed for medical image processing. It provides easy-to-use tools for reading, writing, and processing 3D medical images like MRI scans in formats such as NIfTI.

- **Flask**: Flask is a lightweight Python web framework that helps integrate machine learning models into the web application. It allows us to expose our trained models as RESTful APIs, so the frontend can send input data, receive predictions, and interact with the model in real time. It's a key component for serving AI models to users seamlessly.

6.1.2.3 Database

To ensure efficient data storage and management, a robust database solution was implemented for handling essential user information and system data.

- **PostgreSQL**: An advanced, open-source relational database used for securely storing user data, including doctors and scanners. It ensures reliable, scalable storage with support for complex queries and transactions.

6.1.2.4 Development Tools

Several tools have been employed throughout the development process to ensure smooth collaboration, efficient coding, and rigorous testing. These tools help streamline workflows and improve the overall quality of the project.

- **WebStorm**: AThis powerful IDE is essential for frontend development, especially for working with JavaScript and TypeScript. It enhances our workflow with features like code completion, debugging, and seamless version control integration, allowing us to build the frontend efficiently.

- **IntelliJ IDEA**:As the core IDE for Java development, IntelliJ IDEA supports Spring Boot and other Java frameworks. It provides powerful tools for refactoring, debugging, and running tests, making it a vital part of our backend development process.

- **Postman**: We use Postman to test our RESTful APIs, ensuring smooth communication between the frontend, backend, and AI models by simulating various requests and responses.

- **Git/GitHub**: it is our version control system, helping us manage code changes, track versions, and collaborate smoothly. GitHub hosts our repository, enabling team members to easily contribute and review each

other's work.

Figure 6.1 presents the software architecture diagram, providing a detailed overview of the system's components and their interactions.

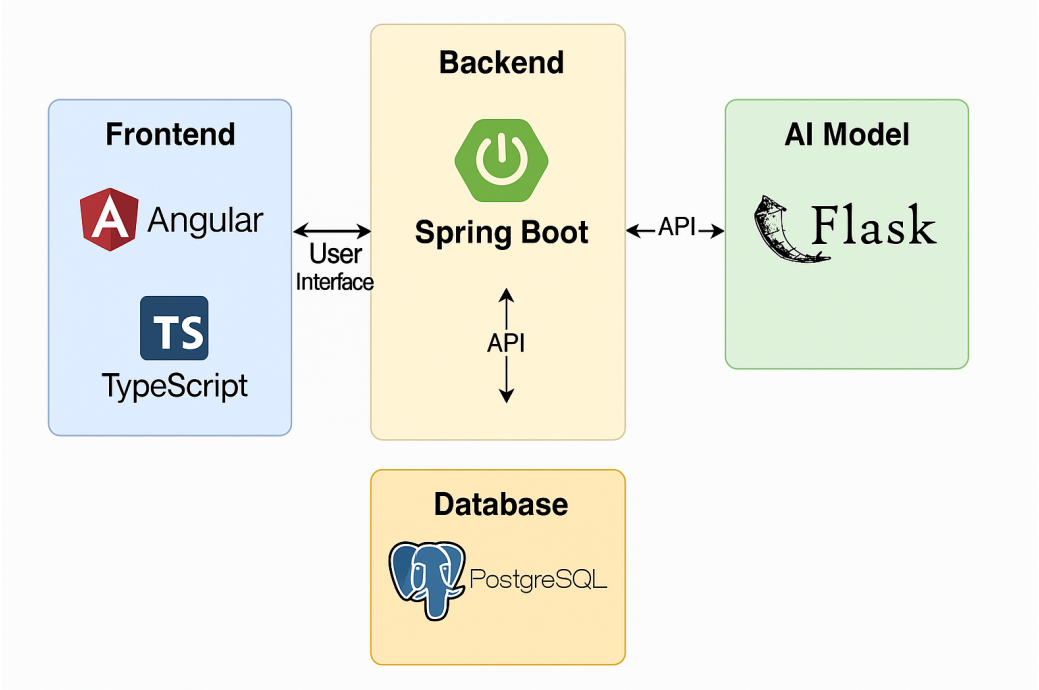


Figure 6.1: Software Architecture Diagram

6.2 Computational results

In this section, we will present the results for both the BraTS 2020 and BraTS 2021 datasets using different configurations and architectures of the model.

6.2.1 Model Performance Using the BraTS 2020 Dataset

Our first approach was to train our base model on the BraTS 2020 dataset. The distribution of the dataset was as follows: 259 training cases, 65 validation cases, and 50 testing cases. Our configuration was standard, with **100 epochs** and 5-fold cross-validation.
The table 6.2 summarizes our results.

Fold	Mean validation dice (%)	Foreground Dice Score(%)	IOU (%)	Sensitivity (%)	Specificity (%)
Fold 1	76.75	73.57	62.4	77.93	99.93
Fold 2	76.23	73.2	62.08	76.53	99.92
Fold 3	72.27	75.00	63.9	79.81	99.93
Fold 4	79.86	74.24	63.24	80.23	99.93
Fold 5	76.76	74.08	63.07	77.99	99.93
Average	**76.37**	**74.01**	**62.93**	**78.49**	**99.93**

Table 6.2: Results for BraTS 2020 Dataset

The overall training duration was 8 hours and 50 minutes, with an average of 59.94 seconds per epoch using the CCK Server.

As a follow-up, we attempted to improve our results by increasing the number of epochs. We trained the model with 250 epochs, the obtained results are summarized in the table 6.3.

Fold	Mean validation dice (%)	Foreground Dice Score(%)	IOU (%)	Sensitivity (%)	Specificity (%)
Necrotic	79.9	77.89	66.2	79.83	99.87
Edema	69.93	70.81	58.21	67.59	99.95
Enhancing Tumor	81.06	75.42	66.14	88.48	99.97
Average	**76.96**	**74.7**	**63.52**	**78.19**	**99.93**

Table 6.3: Results for BraTS 2020 Dataset for 250 epochs

6.2.2 Model Performance Using the BraTS 2021 Dataset

As these results were not satisfactory, we decided to train our model using the BraTS 2021 dataset, which contains more cases compared to BraTS 2020. The dataset containing 1355 cases was distributed as follows: 800 cases for training, 200 for validation, and 355 for testing. We started by training the base model and then explored different strategies to improve the results obtained.

6.2.2.1 nnU-Net Base Model

we trained our model using a 5-fold cross-validation setup, with each fold running for 100 epochs. The table 6.4 summarizes our results:

Fold	Mean validation dice (%)	Foreground Dice Score(%)	IOU (%)	Sensitivity (%)	Specificity (%)
Fold 1	83.38	83.93	76.16	88.39	99.96
Fold 2	83.24	84.13	76.38	88.92	99.96
Fold 3	83.19	83.64	75.88	88.99	99.96
Fold 4	84.35	83.92	76.14	89.12	99.96
Fold 5	84.72	83.43	75.55	88.49	99.96
Average	**83.77**	**83.81**	**76.07**	**88.78**	**99.96**

Table 6.4: Results for BraTS 2021 Dataset

The training lasted 9 hours and 22 minutes, with an average of 60.07 seconds per epoch. The results were overall satisfactory especially compared to results from dataset Brats 2021, but we aimed to improve them by increasing the number of epochs. We trained the model for 250 epochs, the obtained results are summarized in the table 6.5.

Fold	Mean validation dice (%)	Foreground Dice Score(%)	IOU (%)	Sensitivity (%)	Specificity (%)
Necrotic	78.63	79.13	71.18	85.84	99.98
Edema	86.71	86.28	78.04	88.84	99.94
Enhancing Tumor	87.3	87.91	81.09	91.1	99.98
Average	**84.21**	**84.44**	**76.77**	**88.97**	**99.97**

Table 6.5: Results for BraTS 2021 Dataset for 250 epochs

Training this model for 250 epochs lasted 4 hours and 25 minutes.

6.2.2.2 ResEnc nnU-Net

After training the standard nnU-Net model, we proceeded to experiment with the ResEnc nnU-Net. We started with the M-ResEnc configuration, which incorporates residual encoding to enhance feature extraction and improve segmentation performance. We trained it for 250 epochs, the obtained results are summarized in the table 6.6:

Fold	Mean validation dice (%)	Foreground Dice Score(%)	IOU (%)	Sensitivity (%)	Specificity (%)
Necrotic	79.04	79.1	71.34	86.17	99.98
Edema	86.55	86.38	78.21	89.2	99.94
Enhancing Tumor	87.53	87.91	81.08	91.28	99.98
Average	**84.34**	**84.34**	**76.8**	**89.27**	**99.97**

Table 6.6: Results for BraTS 2021 Dataset for 250 epochs with ResEnc

These results show an improvement compared to the standard nnU-Net model. This training lasted 6 hours and 2 minutes, with an average of 81.27 seconds per epoch

6.2.2.3 Models Combination: Max Probability

The results of the previous nnU-Net standard model and ResEnc were excellent, but we aimed to achieve further improvements by applying the max probability approach. This method selects the class with the highest probability from each probability map for every voxel,the obtained results are summarized in the table 6.7.

Fold	Foreground Dice Score(%)	IOU (%)	Sensitivity (%)	Specificity (%)
Necrotic	79.37	71.59	86.43	99.98
Edema	86.57	78.49	89.2	99.94
Enhancing Tumor	87.88	81.21	91.23	99.98
Average	**84.61**	**77.1**	**89.3**	**99.97**

Table 6.7: Results for models combination: Max Probability

6.2.2.4 Models Combination: Average Probability

In this approach, we averaged the probability of each voxel class and classified the voxel with the class having the highest probability, the obtained results are summarized in the table 6.8:

Fold	Foreground Dice Score(%)	IOU (%)	Sensitivity (%)	Specificity (%)
Necrotic	79.7	71.88	86.36	99.98
Edema	86.57	78.49	89.22	99.94
Enhancing Tumor	87.88	81.2	91.21	99.98
Average	**84.71**	**77.19**	**89.3**	**99.97**

Table 6.8: Results for models combination: Average Probability

6.3 Evaluation and Discussion

The evaluation and discussion phase provides insights into the model's performance by analyzing key metrics and trends observed during training. It helps assess the effectiveness of the learning process, identify potential overfitting or underfitting issues, and determine areas for improvement.

6.3.1 Analysis of the Model Evaluation Curve

This section presents an analysis of the model's evaluation curves obtained during training. These curves help assess the model's learning progress, stability, and segmentation performance over time.

6.3.1.1 Model Evaluation Curve

The following image (Figure 6.2) shows the model evaluation curves with the corresponding labels:

- **loss_tr (blue)**: Represents the training loss, indicating how well the model fits the training data.
- **loss_val (red)**: Represents the validation loss, which helps assess the model's generalization to unseen data.

- **pseudo dice (green)**: Represents the Dice score, a common metric for evaluating segmentation performance.
- **pseudo dice (mov. avg.)**: A moving average of the Dice score to smooth out fluctuations during training.

Figure 6.2: Model Evaluation Curve

The training and validation loss curves provide important insights into the learning process of the model. The training loss decreases steadily over time, indicating that the model is effectively learning from the data. Meanwhile, the validation loss stabilizes after a certain number of epochs, showing some oscillations. This suggests that the model has reached a performance plateau, and further improvements may require additional tuning or adjustments in the training strategy.

The pseudo-Dice score increases over time, indicating that the segmentation performance improves as the training progresses. While some fluctuations are observed—mainly due to variations in the validation batches—the overall trend demonstrates a gradual and consistent improvement in the model's ability to accurately segment the images.

6.3.1.2 Epoch Duration Curve

The following image 6.3 show the evolution of the epoch durations where the y-axis represents the time per epoch in seconds, while the x-axis represents the number of epochs. The curve shows how the duration of each epoch evolves over the course of training.

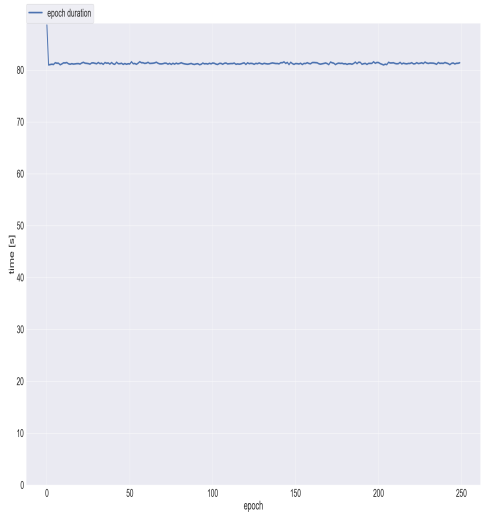


Figure 6.3: Epoch Duration

At the beginning of training, the epoch duration stabilizes quickly around a constant value of approximately 160 seconds. Throughout most of the training process, this duration remains nearly stable, with only minor fluctuations. Toward the final epochs, however, a significant drop in epoch duration is observed, indicating a shift in computational workload.

The overall stable epoch duration suggests a consistent training pipeline, where data loading, augmentation, and model updates are efficiently managed. Minor fluctuations are expected and typically reflect variations in data complexity or processing overhead.

6.3.1.3 Learning Rate Curve

The following image 6.4 show the evolution of the Learning rate where the y-axis represents the learning rate, while the x-axis represents the number of epochs. The curve shows a monotonic decrease in the learning rate as training progresses

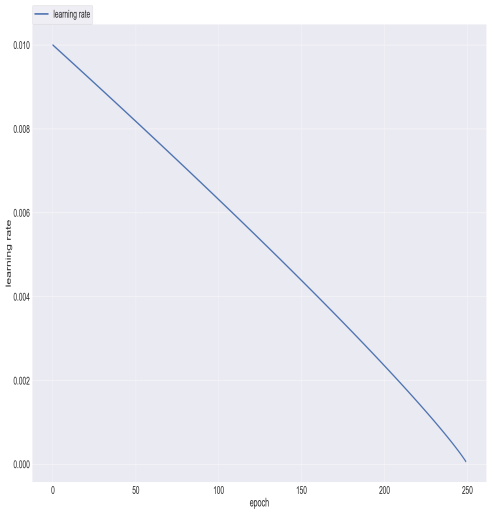


Figure 6.4: Learning Rate

The learning rate starts at a relatively high value (around 0.01) and decreases progressively throughout the training process. This reduction follows a smooth and continuous schedule, reaching a near-zero value by the final epochs.

The purpose of decreasing the learning rate is to balance exploration and fine-tuning during training. A higher learning rate at the beginning allows the

model to explore the parameter space more broadly with larger updates. As training progresses, reducing the learning rate enables smaller, more precise adjustments, helping the model converge and avoid overshooting the optimal solution.

6.3.2 Model Performance Comparison

In this subsection, we evaluate the model's performance using different datasets and approaches, with a focus on the **Foreground Dice Score** and **IoU**. First, we compare our results internally by testing different model configurations and combination strategies. Then, we compare our best results with state-of-the-art methods in brain tumor segmentation.

6.3.2.1 Internal Comparison

This section focuses on comparing our different approaches to identify the best results. The table 6.9 summarizes our findings.

Dataset	**Model**	**Foreground Dice Score(%)**	**Evolution of Dice Score(%)**	**IoU** (%)	**Evolution of IoU(%)**
BraTS 2020	Standard nnU-Net	74.7	–	63.52	–
BraTS 2021	Standard nnU-Net	84.44	+9.74	76.77	+13.52
	ResEnc nnU-Net	84.34	-0.1	76.8	-0.03
	Combined Approach: Max Probability	84.71	+0.27	77.19	+0.62
	Combined Approach: Average Probability	84.61	-0.1	77.1	-0.09

Table 6.9: Comparison of segmentation performance across different models and datasets

Training the model with the BraTS 2021 dataset proved to be a significant

improvement in terms of metrics. The model is now capable of segmenting images with better accuracy. The approach of combining two models also showed a small improvement in Dice score and IoU, helping to refine the segmentation slightly.

6.3.2.2 External Comparison

In this section, we compare our best approach with state-of-the-art models using the BraTS 2021 dataset, focusing only on the Foreground Dice Score due to its widespread use in other research. Table 6.10 summarizes the results.

Researcher	**Model**	**Dice Score (%)**	**Dice Difference (%)**
Our Approach	Prob. Averaging	84.71	–
Zhang et al.	AugTransU-Net	93.6	-8.89
ZongRen et al.	DenseTrans	93.2	-8.94
Lin et al.	CKD-TransBTS	92.46	-7.75

Table 6.10: Comparison with State-of-the-Art Models Based on Dice Score

The results show that while our approach achieves a solid Dice score of 84.71%, it still lags behind state-of-the-art models such as CKD-TransBTS and AugTransU-Net. However, our method remains competitive considering its simplicity and interpretability. The gap highlights opportunities for further improvement, such as integrating more advanced architectures or training strategies to enhance segmentation accuracy.

6.3.3 Qualitative Analysis

In this subsection, we compare the predicted images with the ground truth to demonstrate the efficacy of the model in segmenting the different tumor classes. The comparison includes the various approaches used in this analysis.

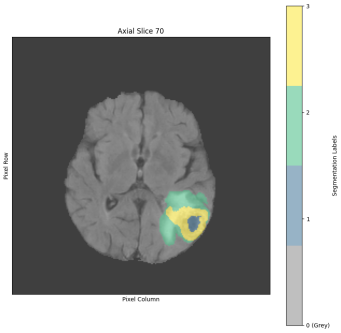


Figure 6.5: Groud Truth

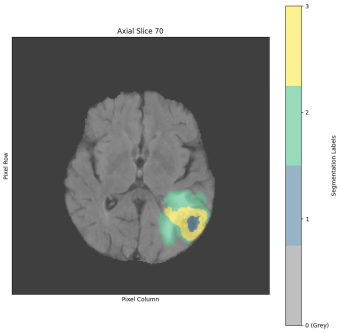


Figure 6.6: Standard nnU-Net

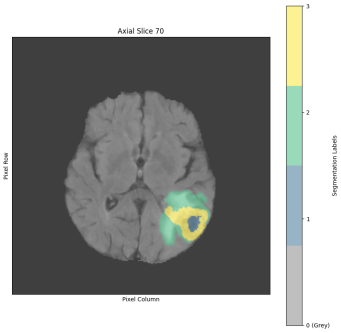


Figure 6.7: ResEnc

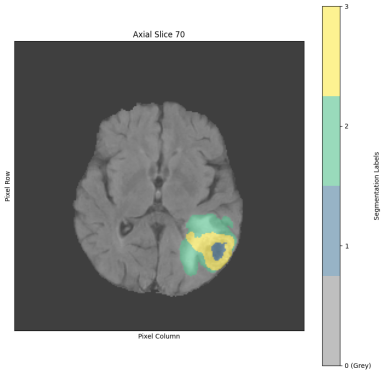


Figure 6.8: Combination:Average

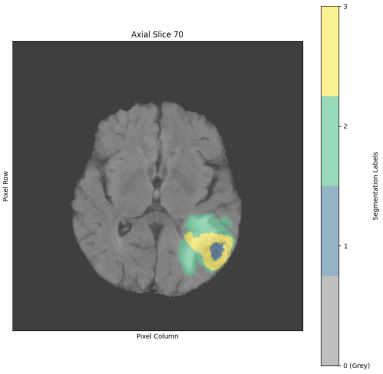


Figure 6.9: Combination:Max

- **Class 0:** Background

- **Class 1:** Necrotic (NCR) & Non-Enhancing Tumor (NET)
- **Class 2:** Peritumoral Edema (ED)
- **Class 3:** Enhancing Tumor (ET)

All the predictions are very close to the ground truth, with only slight differences. The combined approach shows slight refinement and improvement in segmentation compared to other standard models.

6.3.4 Limitations of the Model

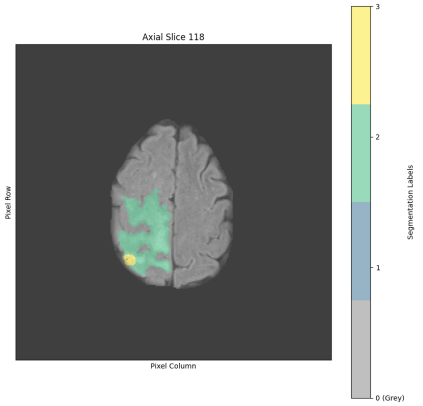
While the model shows great promise in segmenting brain tumors, with a Dice score over 84%, its metrics are not close to state-of-the-art models. After inspecting the result files and additional segmented images, we found that the differences in Dice scores are caused by:

- **Misidentifying tumor classes:**

In some delicate cases, the model classified certain voxels as a class that is not present in the image. This misidentification significantly reduces the Dice score, as in these cases, the Dice score for the misidentified class is counted as 0 or close to 0.

- **Inability to identify small regions:**

In some cases, certain classes are represented by only a few voxels, making them difficult to identify. In other cases, this leads to incomplete segmentation of the region.



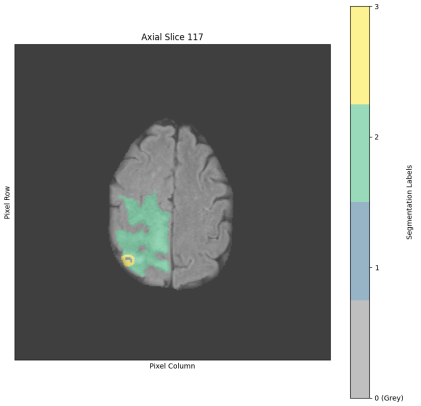


Figure 6.10: Segmentation Image Figure 6.11: Ground Truth

Figure 6.12: Limitation of the model

In the figure 6.12, we observe the difficulty of the model in completely segmenting the Necrotic & Non-Enhancing Tumor (class 1) due to its small area.

6.4 Graphical Interface

To ensure a smooth and role-based user experience, we developed a user-friendly graphical interface for our brain tumor segmentation platform, **MindScan**. The platform is designed for medical professionals and administrators, supporting the diagnostic workflow and system management through the following pages:

- **Home Page :** Introduces users to MindScan and its AI-based functionality for brain tumor segmentation. It features intuitive navigation to access core parts of the platform. Figure 6.13 shows the layout and key elements of the home page interface.

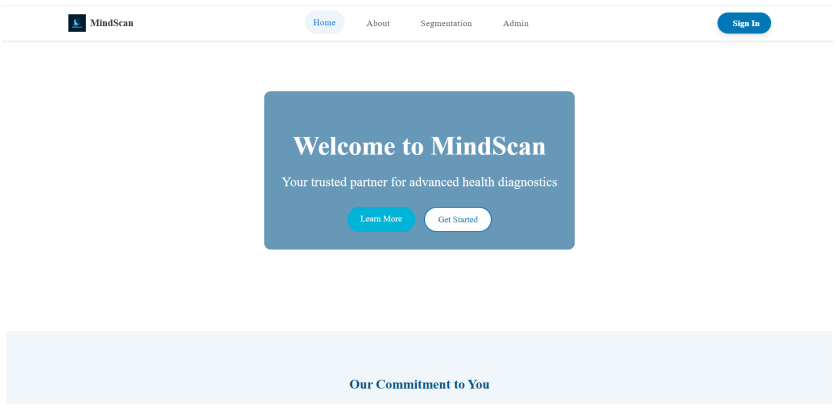


Figure 6.13: Home Page

- **About Page :** Provides an overview of the project's motivation and technologies. It also includes:
  - **Results and Discussion :** Summarized metrics demonstrating the model's effectiveness.
  - **Meet the Team :** Highlights the developers behind MindScan.

Figure 6.14 shows the layout and key elements of the about page interface.

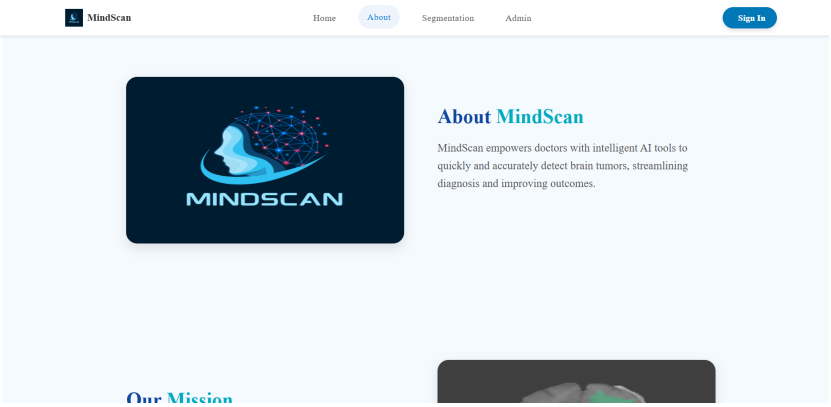


Figure 6.14: About Page

- **Login and Register Pages :** Built with Spring Boot, these pages provide secure access for scanners, doctors, and administrators. After logging in:
  - **Doctors :** Doctors must enter their unique ID to access the system.
  - **Scanners :** Scanners need to provide both their own ID and the doctor's ID to whom they will send the processed data.
    Figure 6.15 , Figure 6.16 and Figure 6.17 shows the layout and key elements of the Login and Register Pages interface and Login Page for Entering Scanner and Doctor ID.

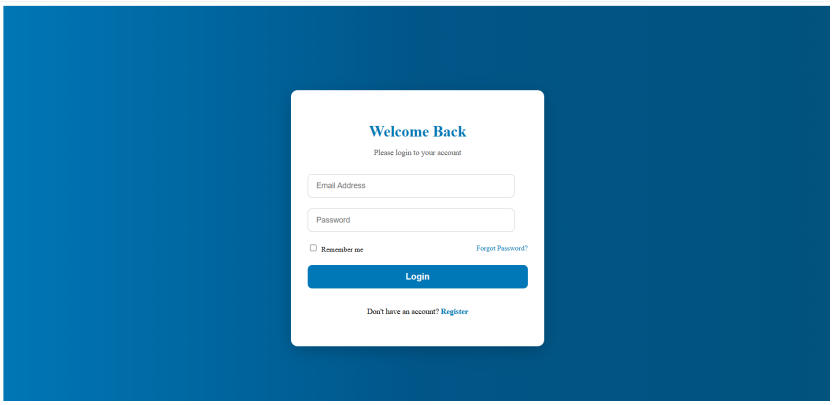


Figure 6.15: Login Page

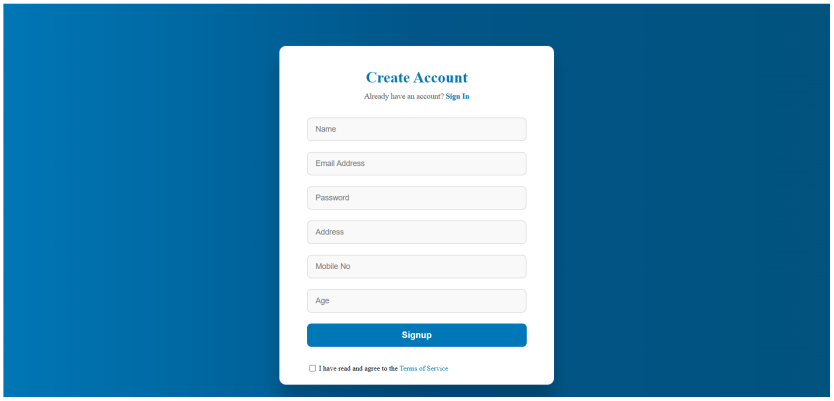


Figure 6.16: Register Page

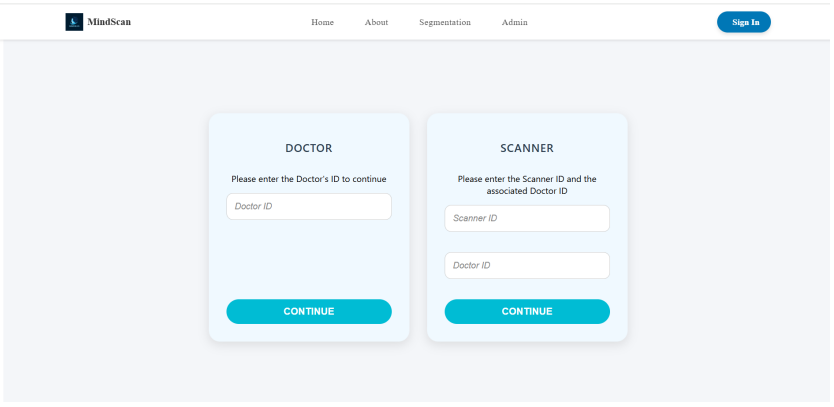


Figure 6.17: Login Page for Entering Scanner and Doctor ID

- **Segmentation Page :** Core page for processing and visualizing brain MRI segmentations.
  Users:
  - Upload four MRI modalities (T1, T1ce, T2, FLAIR in NIfTI format)
  - Launch segmentation via our model
  - View the results with overlays in axial, coronal, or sagittal views
  - Navigate through slices using a slider
    Figure 6.18 shows the layout and key elements of the segmentation Page interface.

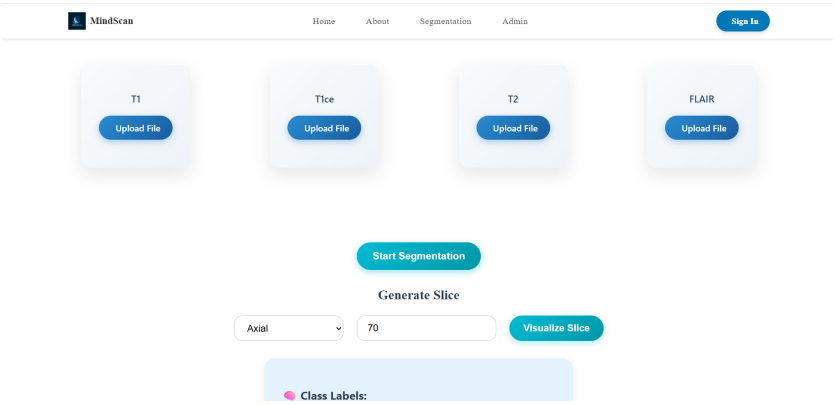


Figure 6.18: Segmentation Page

Before accessing the **Admin Page**, users must first log in as an administrator. This is done through a secure Admin Login page, which ensures that only authorized administrators can access the system's management tools.

- **Admin Login Page :** Admins enter their credentials to gain access to the system's administrative functionalities.

Figure 6.19 shows the layout and key elements of the admin login Page interface .

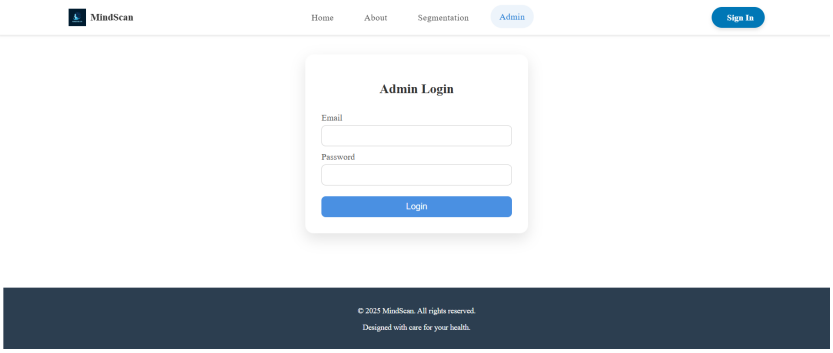


Figure 6.19: Admin Login Page

- **Admin Page :** Allows administrators to manage the platform efficiently.
  Main functionalities include:
  - **Segmentation Dashboard :** View model performance, scanners summary, and doctors summary
  - **User Management :** Edit, and delete users
    Figure 6.20 shows the layout and key elements of the admin dashboard Page interface .

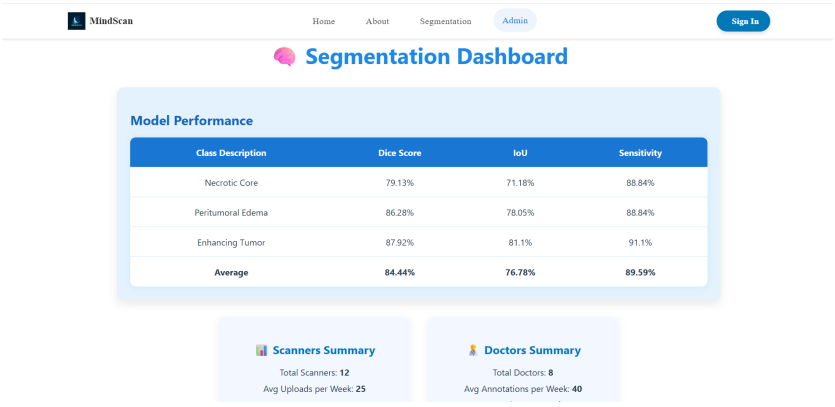


Figure 6.20: Admin Page –Dashboard

Figure 6.21 shows the layout and key elements of the admin user management Page interface .

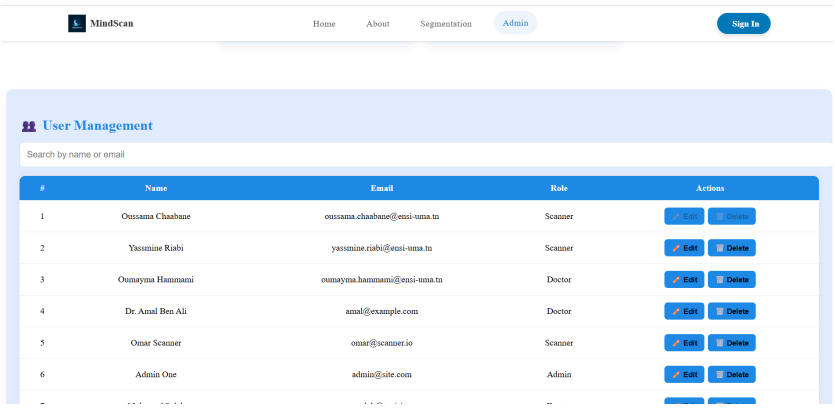


Figure 6.21: Admin Page – User Management

- **Contact Page :** A simple form enables users to send feedback, report issues, or request support.

  Figure 6.22 shows the layout and key elements of the contact Page interface .

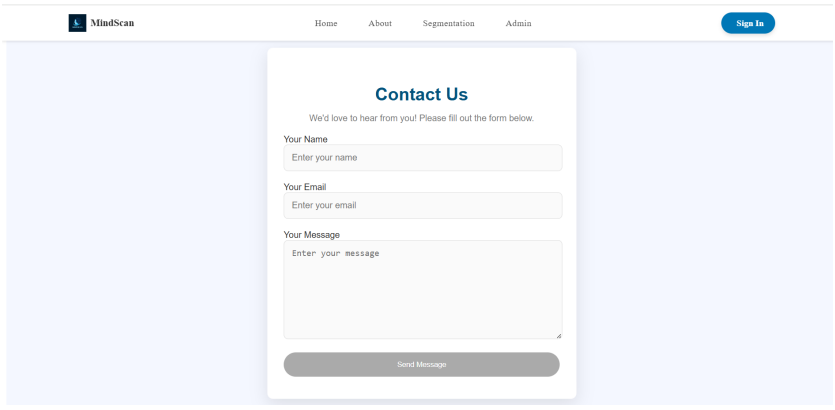


Figure 6.22: Contact Page

Chapter 7

Conclusion

This project demonstrates the potential of deep learning, specifically fine-tuned nnU-Netv2 models, in improving the accuracy and reliability of brain tumor segmentation. Through experimentation with different configurations and model combinations, we achieved promising results on the BraTS 2020 and 2021 datasets. By integrating a user-friendly interface, we aimed to make this tool accessible to healthcare professionals and contribute to earlier diagnosis.

Our work highlights how AI can assist the medical field and lays the groundwork for future improvements, such as expanding the system to other types of medical image analysis. Moving forward, we plan to apply our segmentation approach to other organs, with the long-term goal of developing a comprehensive full-body tumor segmentation solution.

To enhance usability, we also aim to streamline the workflow between scanner operators and doctors by simplifying the image upload process and facilitating appointment scheduling.

Bibliography

[1] NVIDIA. nnu-net for tensorflow2, 2024. URL https://catalog.ngc.nvidia.com/orgs/nvidia/teams/dle/resources/nnunet_tf2.

[2] ResearchGate. URL https://www.researchgate.net/publication/335233055_Multi-step_Cascaded_Networks_for_Brain_Tumor_Segmentation.

[3] Bjoern H. Menze et al. Sample mri of brain tumor segmentation from the brats21 challenge 2021 dataset. https://www.researchgate.net/figure/Sample-MRI-of-brain-tumor-segmentation-BraTS21-Challenge-2021-dataset-a-Norm fig2_375283507, 2021. Accessed: 2025-05-06.

[4] 72 must-know brain tumor statistics (2024). URL https://www.aaroncohen-gadol.com/en/patients/brain-tumor/types/statistics#:~:text=Additional%20Brain%20Tumors%20Statistics,-45.&text=About%2025%25%20of%20initial%20brain,survival%20duration%20of%2015%20months.

[5] Brain tumor segmentation using ai. URL MRI(MagneticResonanceImaging).

[6] Geert Litjens, Thijs Kooi, Babak Ehteshami Bejnordi, Arnaud Arindra Adiyoso Setio, Francesco Ciompi, Mohsen Ghafoorian,

Jeroen AWM van der Laak, Bram van Ginneken, and Clara I Sánchez. A survey on deep learning in medical image analysis. *Medical image analysis*, 42:60–88, 2017.

[7] Mauricio Reyes, Rafael Meier, Sergio Pereira, Carlos A Silva, Friedrich-Michael Dahlweid, Hermann von Tengg-Kobligk, Patrick Vollmuth, and Roland Wiest. Interpretability of deep learning models in medical imaging: a survey. *Medical Image Analysis*, 65:101729, 2020.

[8] Todd C Hollon, Brenden Pandian, Anand R Adapa, Edward Urias, Alexander V Save, Sadhana S Khalsa, Daniel G Eichberg, Rohan S D'Amico, Zain Farooq, Shayna Lewis, et al. Near real-time intraoperative brain tumor diagnosis using stimulated raman histology and deep neural networks. *Nature medicine*, 26(1):52–58, 2020.

[9] CGB Yogananda, Neha J Shah, Noushin Beig, Hrishikesh Patel, Kalpana Bera, Jasmeet Suri, Nicole Braman, Marcelo Pinho, Thakur Pallavi, Gourav Shukla, et al. A fully automated deep learning network for brain tumor segmentation. *Tomography*, 6(2):186–193, 2020.

[10] K Robin Yabroff, Jennifer Lund, Deanna Kepka, and Angela Mariotto. Economic burden of cancer in the us: Estimates, projections, and future research. *Cancer Epidemiology, Biomarkers Prevention*. URL `https://pmc.ncbi.nlm.nih.gov/articles/PMC3191884/`.

[11] Fabian Isensee, Paul F Jaeger, Simon AA Kohl, Jens Petersen, and Klaus H Maier-Hein. nnu-net: a self-configuring method for deep learning-based biomedical image segmentation. *Nature methods*, 18(2): 203–211, 2021.

[12] Mohamed Skander Feyjie, Dwarikanath Mahapatra, Christos Sakaridis, Jean-Philippe Thiran, and Mauricio Reyes. Semi-supervised few-shot learning for medical image segmentation. In *Proceedings of the*

IEEE/CVF Conference on Computer Vision and Pattern Recognition, pages 10947–10956, 2021.

[13] Mohammed Zakariah, Muna Al-Razgan, and Taha Alfakih. Dual vision transformer-dsunet with feature fusion for brain tumor segmentation. *Heliyon*, 10(18):e37804, 2024. ISSN 2405-8440. doi: https://doi.org/10.1016/j.heliyon.2024.e37804. URL `https://www.sciencedirect.com/science/article/pii/S2405844024138352`.

[14] Salha M. Alzahrani and Abdulrahman M. Qahtani. Knowledge distillation in transformers with tripartite attention: Multiclass brain tumor detection in highly augmented mris. *Journal of King Saud University - Computer and Information Sciences*, 36(1):101907, 2024. ISSN 1319-1578. doi: https://doi.org/10.1016/j.jksuci.2023.101907. URL `https://www.sciencedirect.com/science/article/pii/S1319157823004615`.

[15] Jinglin Yuan. Brain tumor image segmentation method using hybrid attention module and improved mask rcnn. *Scientific Reports*, 14(1):20615, Sep 2024. ISSN 2045-2322. doi: 10.1038/s41598-024-71250-4. URL `https://doi.org/10.1038/s41598-024-71250-4`.

[16] Yin Li, Kaiyi Zheng, Shuang Li, Yongju Yi, Min Li, Yufan Ren, Congyue Guo, Liming Zhong, Wei Yang, Xinming Li, and Lin Yao. A transformer-based multi-task deep learning model for simultaneous infiltrated brain area identification and segmentation of gliomas. *Cancer Imaging*, 23(1):105, Oct 2023. ISSN 1470-7330. doi: 10.1186/s40644-023-00615-1. URL `https://doi.org/10.1186/s40644-023-00615-1`.

[17] Bo Ma, Qian Sun, Ze Ma, Baosheng Li, Qiang Cao, Yungang Wang, and Gang Yu. Dtasunet: a local and global dual transformer

with the attention supervision u-network for brain tumor segmentation. *Scientific Reports*, 14(1):28379, Nov 2024. ISSN 2045-2322. doi: 10.1038/s41598-024-78067-1. URL https://doi.org/10.1038/s41598-024-78067-1.

[18] Jianwei Lin, Jiatai Lin, Cheng Lu, Hao Chen, Huan Lin, Bingchao Zhao, Zhenwei Shi, Bingjiang Qiu, Xipeng Pan, Zeyan Xu, Biao Huang, Changhong Liang, Guoqiang Han, Zaiyi Liu, and Chu Han. Ckd-transbts: Clinical knowledge-driven hybrid transformer with modality-correlated cross-attention for brain tumor segmentation. *IEEE Transactions on Medical Imaging*, 42(8):2451–2461, 2023. doi: 10.1109/TMI.2023.3250474.

[19] Li ZongRen, Wushouer Silamu, Wang Yuzhen, and Wei Zhe. Densetrans: Multimodal brain tumor segmentation using swin transformer. *IEEE Access*, 11:42895–42908, 2023. doi: 10.1109/ACCESS.2023.3272055.

[20] Muqing Zhang, Dongwei Liu, Qiule Sun, Yutong Han, Bin Liu, Jianxin Zhang, and Mingli Zhang. Augmented transformer network for mri brain tumor segmentation. *Journal of King Saud University - Computer and Information Sciences*, 36(1):101917, 2024. ISSN 1319-1578. doi: https://doi.org/10.1016/j.jksuci.2024.101917. URL https://www.sciencedirect.com/science/article/pii/S1319157824000065.

[21] Meryem Altin Karagoz, O. Ufuk Nalbantoglu, and Geoffrey C. Fox. Residual vision transformer (resvit) based self-supervised learning model for brain tumor classification. 2024. URL https://arxiv.org/abs/2411.12874.

[22] Fabian Isensee et al. nnu-net: Self-adapting framework for u-net-based medical image segmentation. *arXiv preprint arXiv:2101.03147*, 2021.